

Machine Learning

6. *Supervised Learning* 3

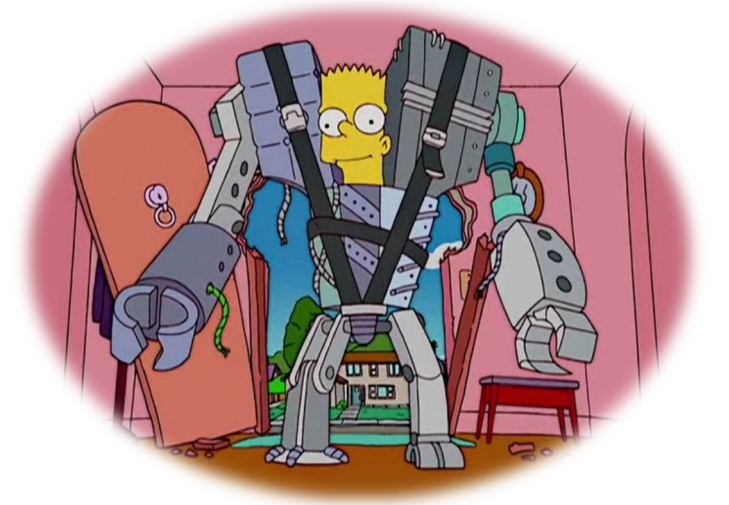
Sunwoo Lee
sunwool@inha.ac.kr



INHA UNIVERSITY

Today, we will learn...

- Supervised Learning 3
 - What is Support Vector Machine (SVM)?
 - Why is SVM useful?
 - General applicability of SVM (Kernel Method)



Do not share or distribute these slides!

저작권 안내 / Copyright Notice

본 사이트에서 수업 자료로 이용되는 저작물은 저작권법 제25조 수업목적저작물 이용 보상금제도의 의거, 한국복제전송저작권협회와 약정을 체결하고 적법하게 이용하고 있습니다. **약정범위를 초과하는 사용은 저작권법에 저촉될 수 있으므로 수업자료의 대중 공개·공유 및 수업 목적외의 사용을 금지합니다. (일부 공개로 제공되는 유튜브 강의를 불법으로 공유하거나 배포하는 행위를 금함)**

The teaching materials provided on this site are copyrighted and signed legally by the Korea Reproduction and Transmission Rights Association. Using these materials beyond the scope of the agreement is a violation against copyright laws. **Hence, one is not allowed to open or share the materials in public. Using them outside of the class is strictly prohibited.**

2020. 3. 11.

인하대학교·한국복제전송저작권협회

Inha University · Korea Reproduction and Transmission Rights Association

- ***Unfortunately, predictions can be wrong...***
- Support Vector Machine (SVM)
- Kernel Method

Why Might Predictions be Wrong? (1/3)

- True non-determinism
 - E.g., consider flipping a coin.
 - $p(\text{heads}) = \theta$
 - Our model will estimate θ
 - If $\theta > 0.5$, our model predict 'heads', otherwise 'tails'.
- The core principle behind Machine Learning is:
 - Train our models on the observations.
 - However, the output of our models will not be the same even when the input is the same!
 - Thus, we just **hope** the model would do its best in expectation.



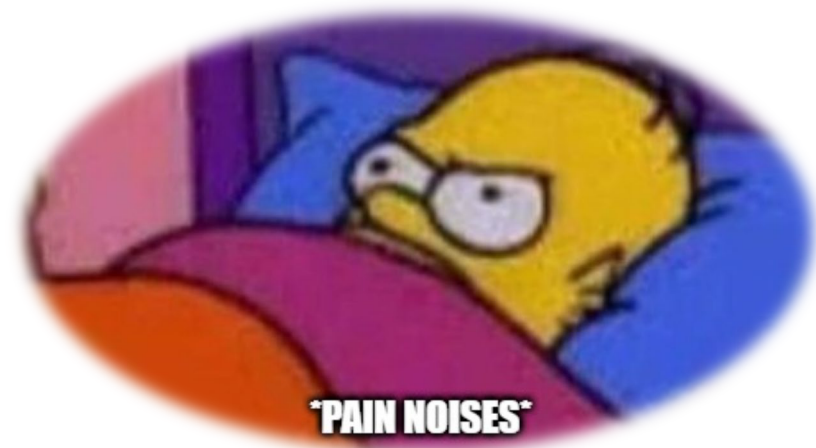
Why Might Predictions be Wrong? (2/3)

- Partial Observability

- Something needed to predict y is missing from the observations x .
- E.g., N -bit parity problem
 - What if x contains only $N - 1$ bits (hard partial observability)?
 - x contains N bits but learners may ignore some of them for reasons (soft partial observability).

- Noise in the observation x

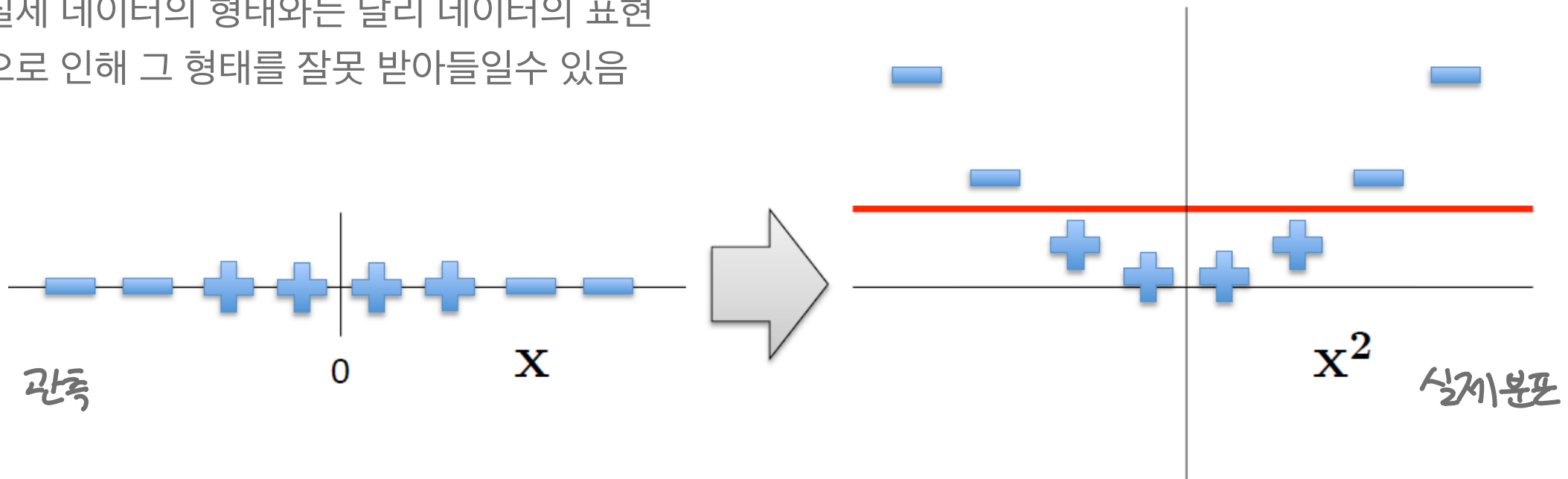
- Measurement errors
- Instrument limitations
- Etc.



Why Might Predictions be Wrong? (3/3)

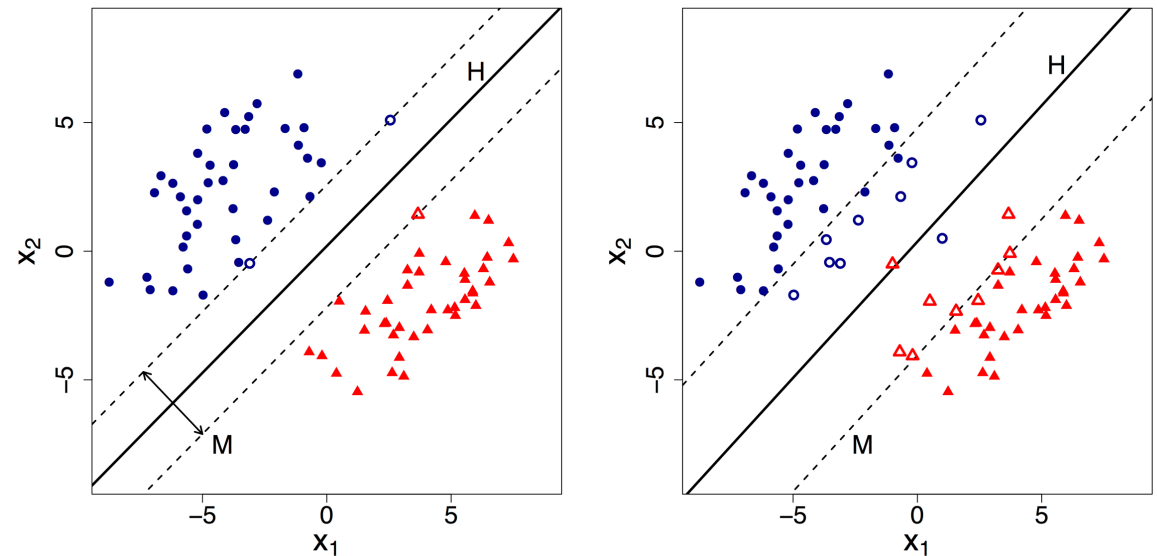
- Representational Bias
 - Having right features (x) is crucial!

실제 데이터의 형태와는 달리 데이터의 표현
으로 인해 그 형태를 잘못 받아들일수 있음



In this lecture, we will specifically focus on this reason!

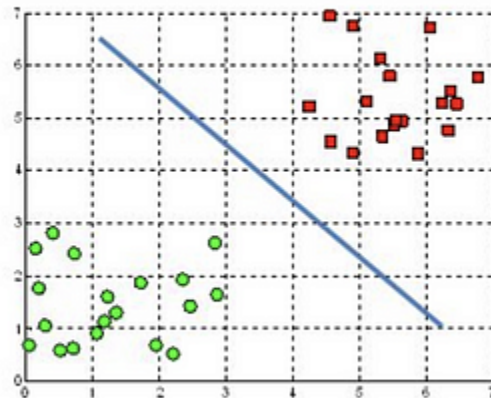
- Unfortunately, predictions can be wrong...
- **Support Vector Machine (SVM)**
 - **What is it?**
 - Algorithm development
- Kernel Method



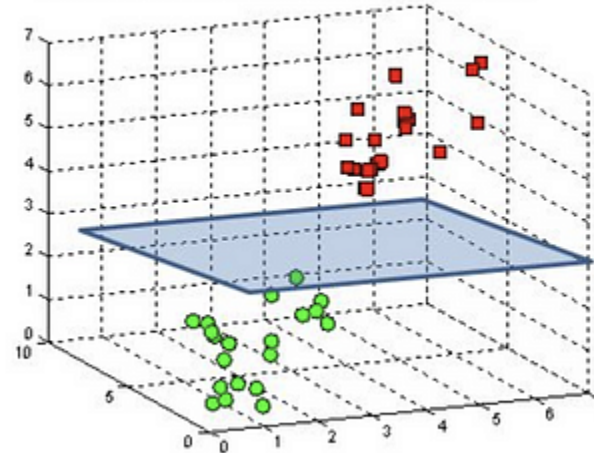
Problem Statement

- The objective here is to find a hyperplane in an N-dimensional space that distinctly classifies the data points.
- Specifically, we want to **maximize the margin**, i.e. the maximum distance between data points of both classes.

A hyperplane in $\mathbb{R}^2$ is a line



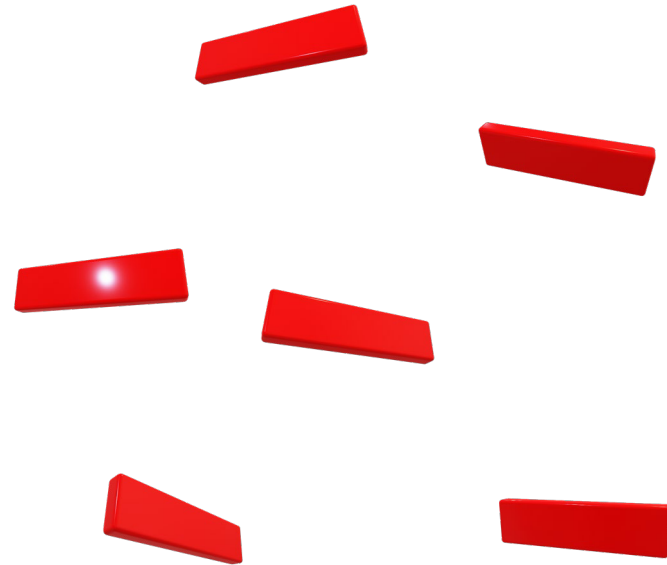
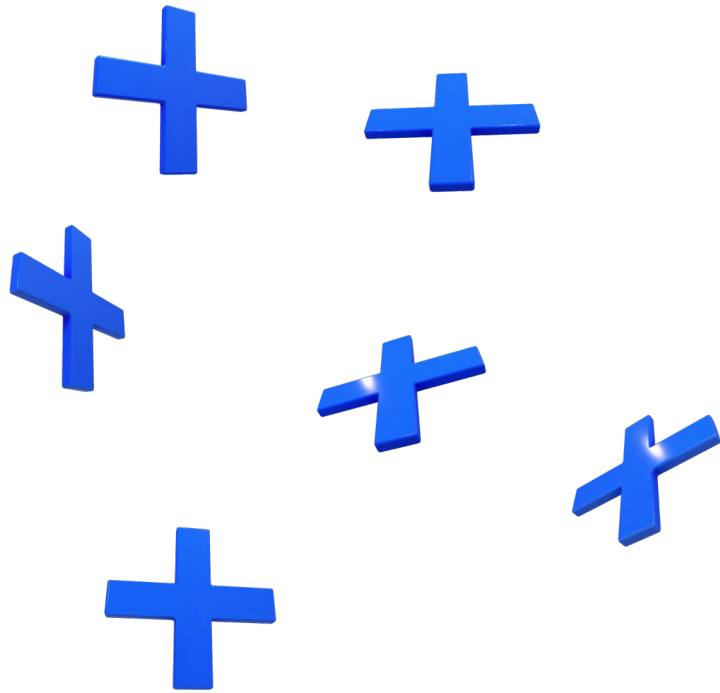
A hyperplane in $\mathbb{R}^3$ is a plane



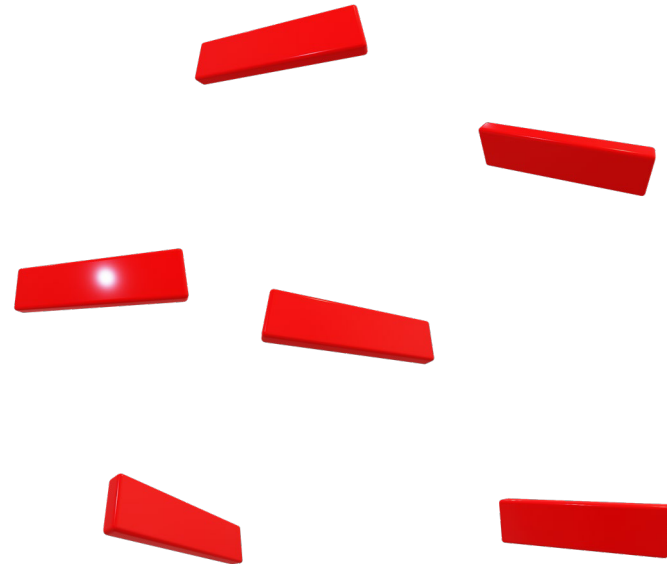
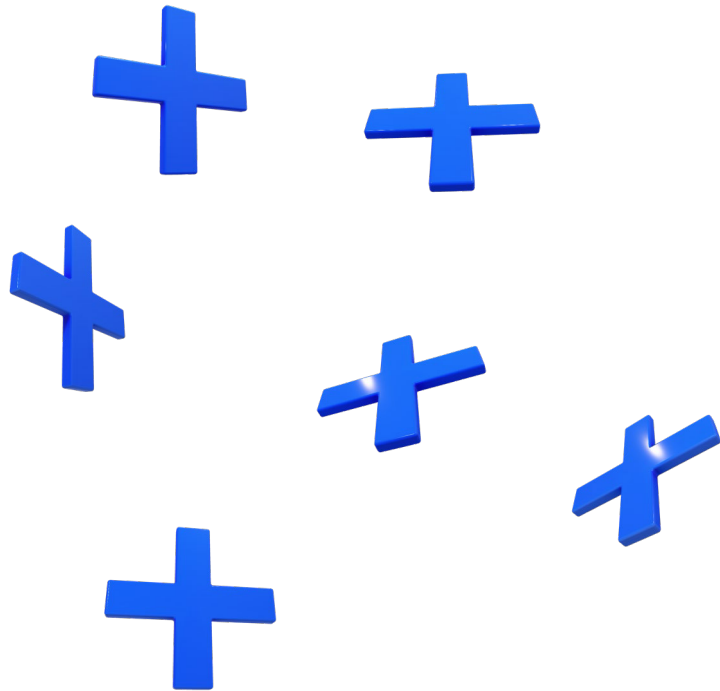
Why is SVM Good?

- [**Versatile**] Generally used for both classification and regression.
- [**Linearity**] Works well for both linear and non-linear problems.
- [**Well-Studied**] Has a strong theoretical foundation.
- [**Robust**] Relatively good performance can be expected only with few training data. = 이상치의 영향을 적게 받음 $\longleftrightarrow$ 결정트리
이상치 영향 매우 ↑

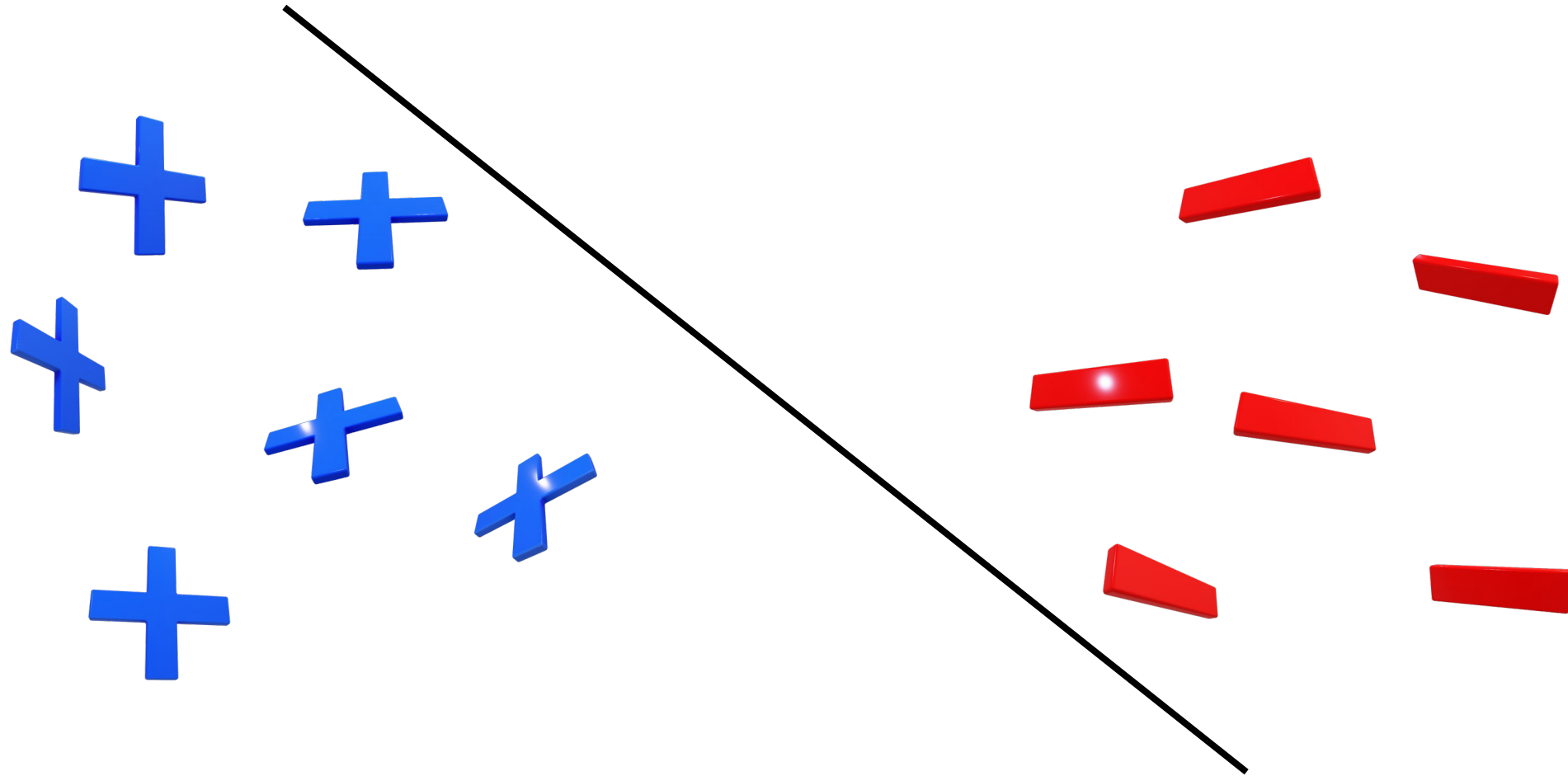
Intuitions



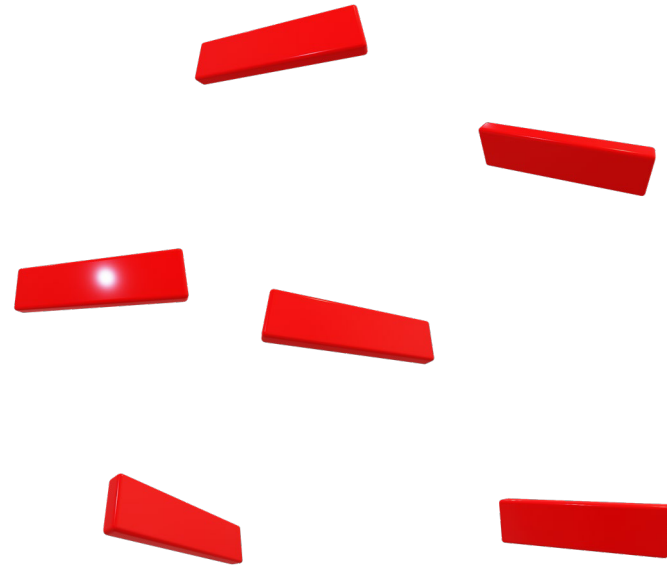
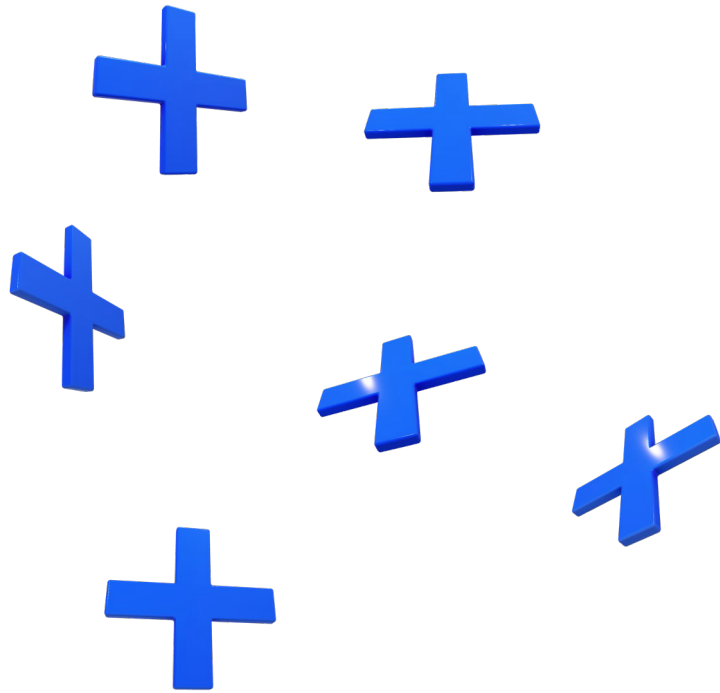
Intuitions



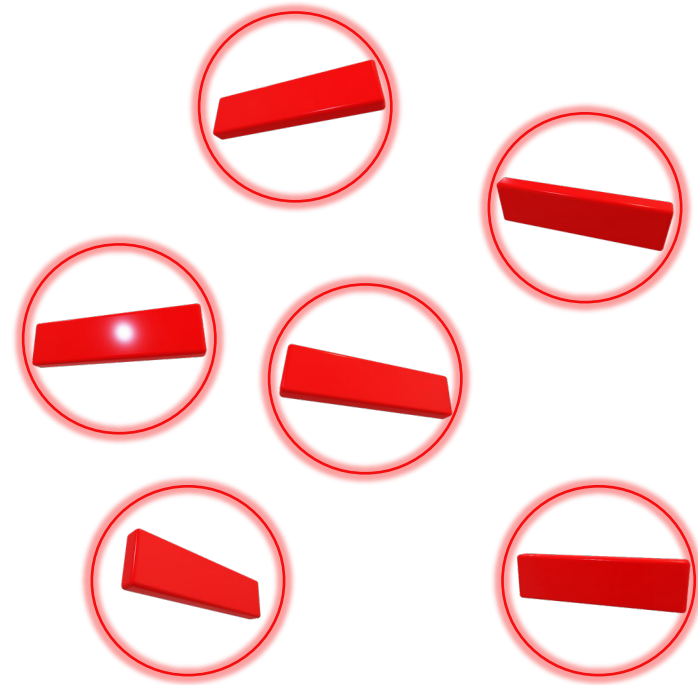
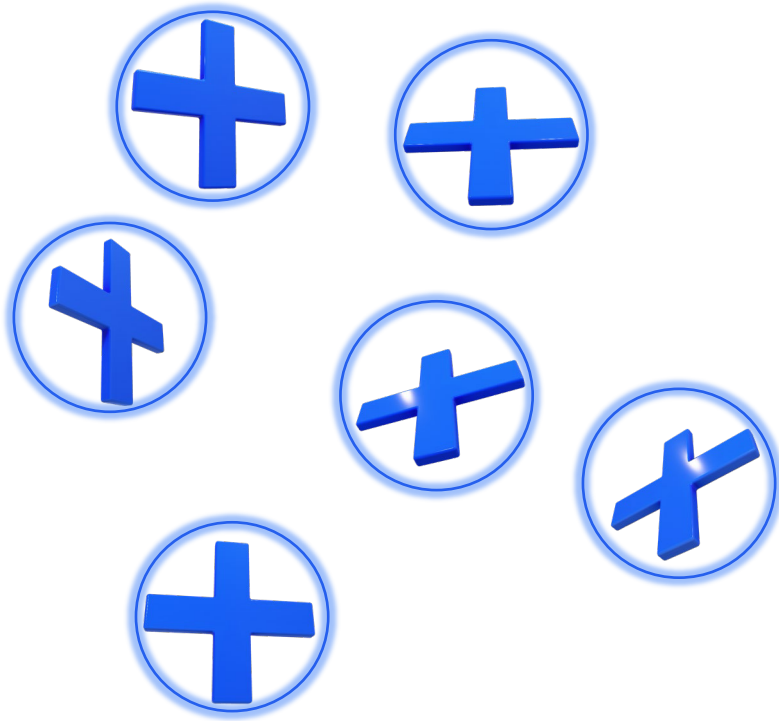
Intuitions



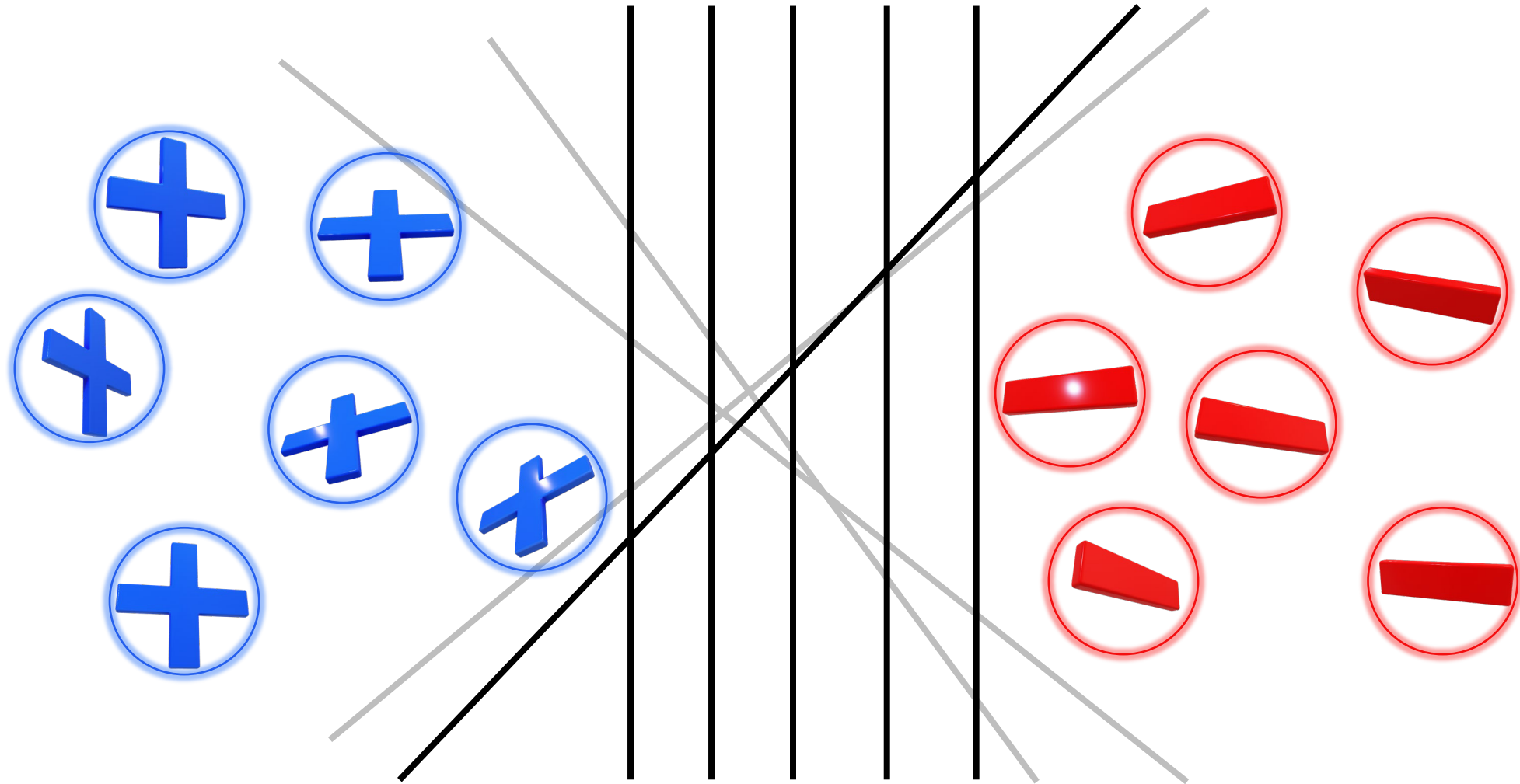
A 'Good' Separator



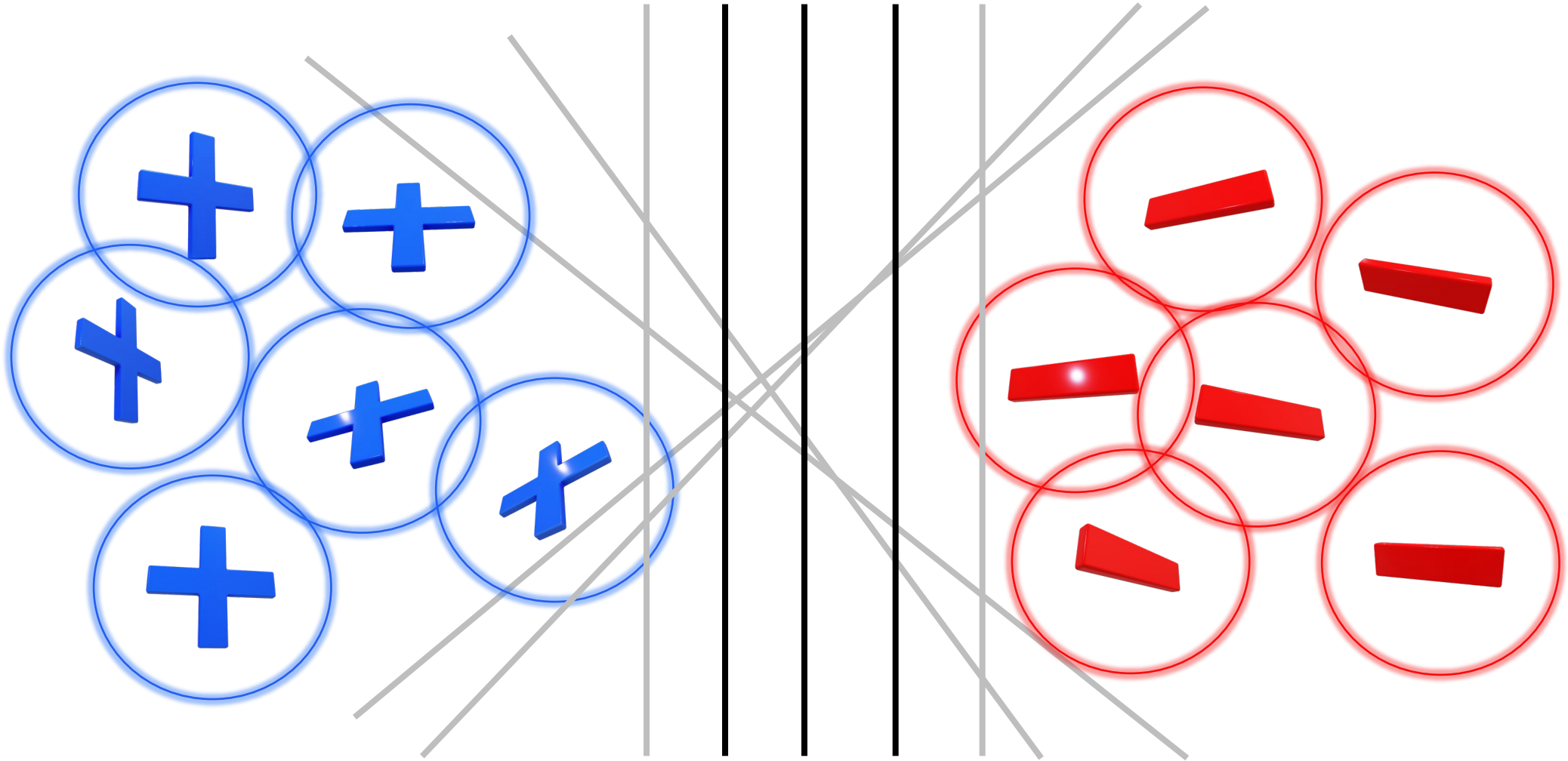
Let Us Assume Each Data Has Noise



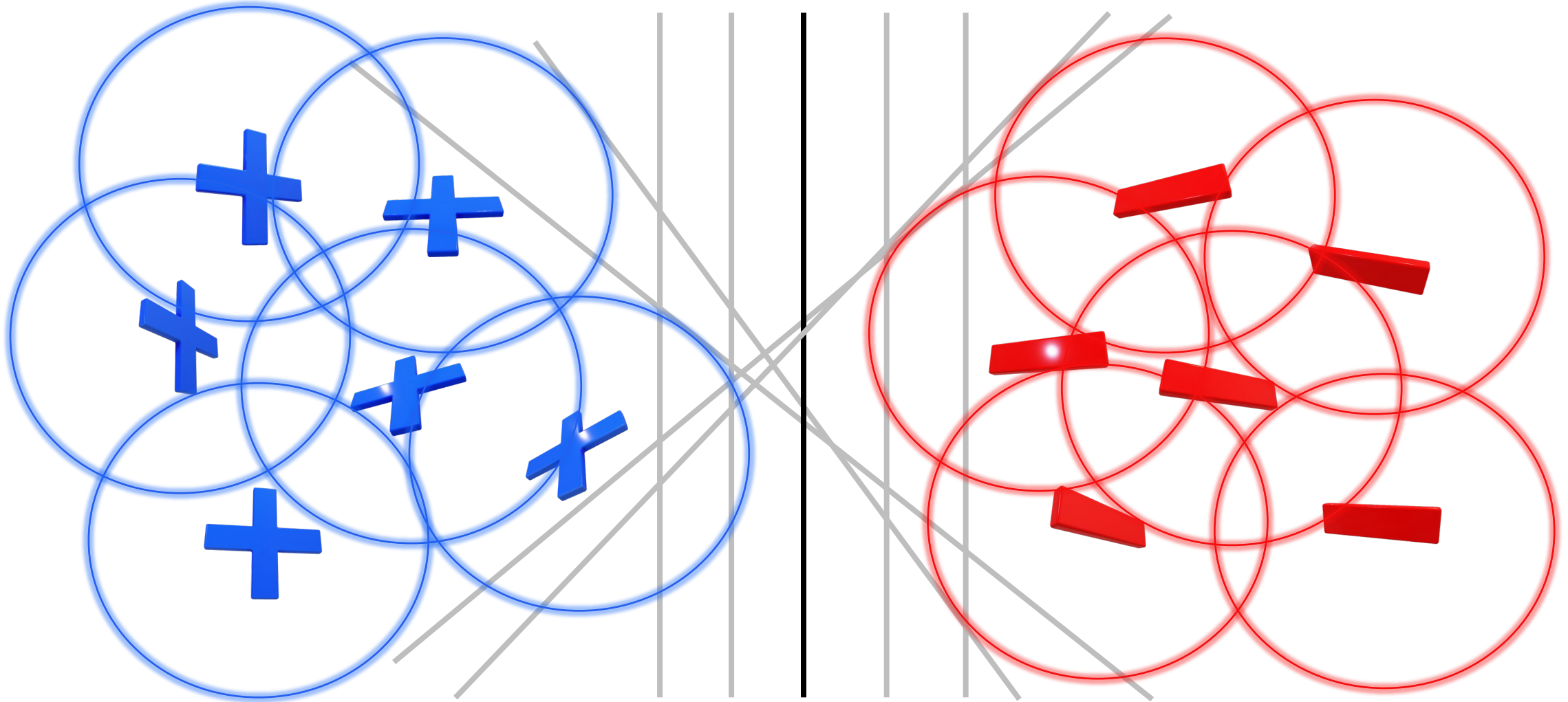
Some Possible Separators Will be Ruled Out



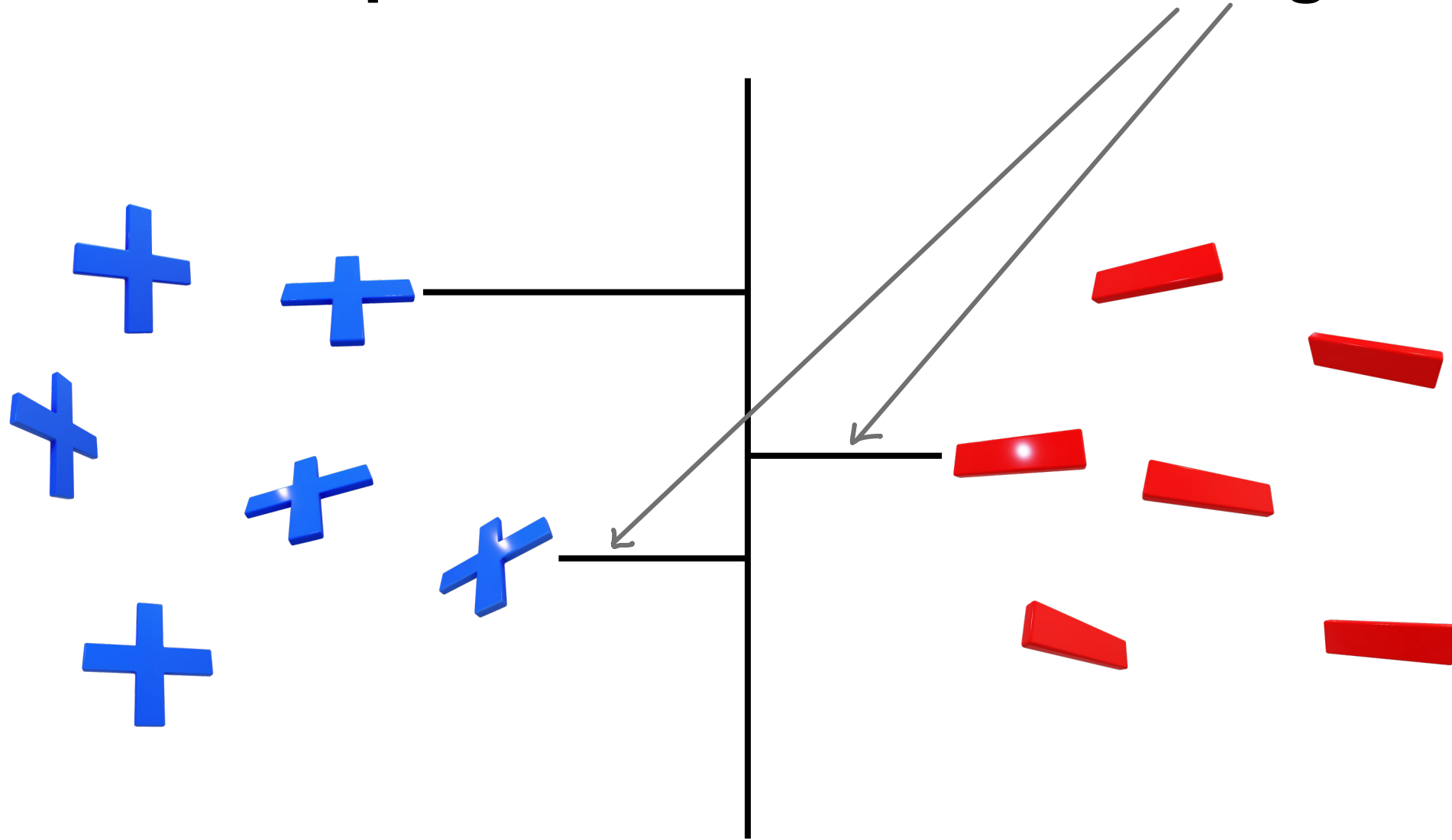
What If the Noise Gets Larger?



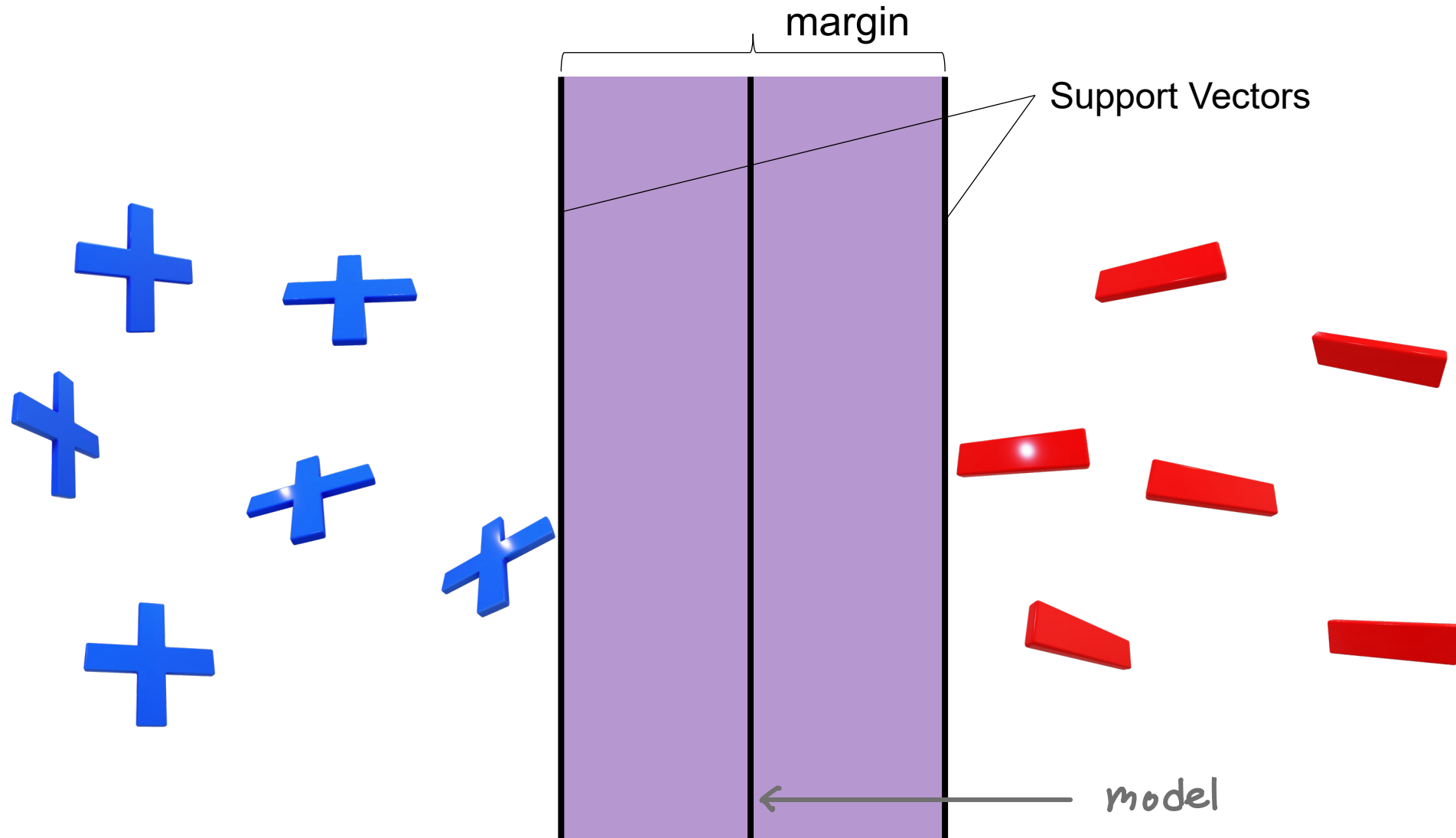
It Will End Up Having Only One Separator!



This Separator Maximizes the Margin

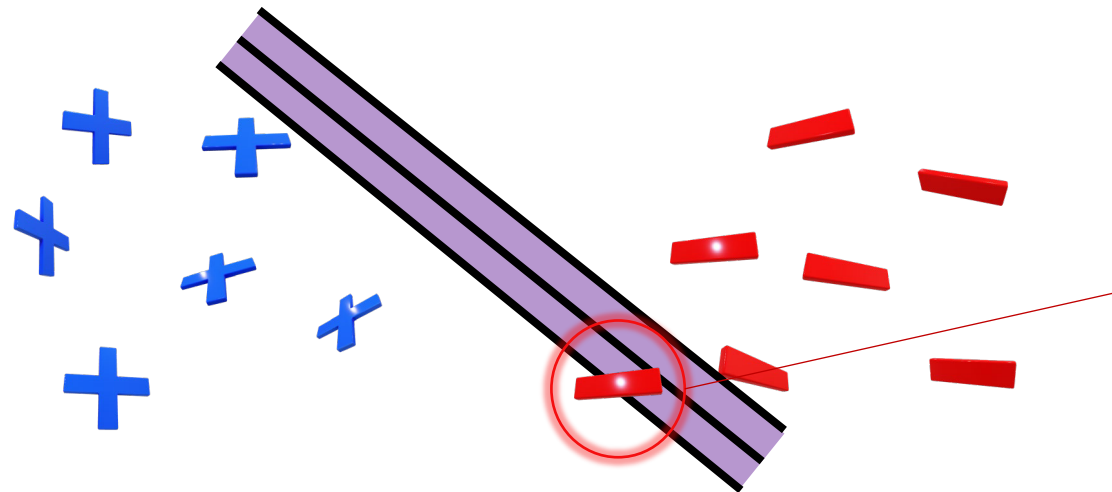


'Fat' Separators are Good!



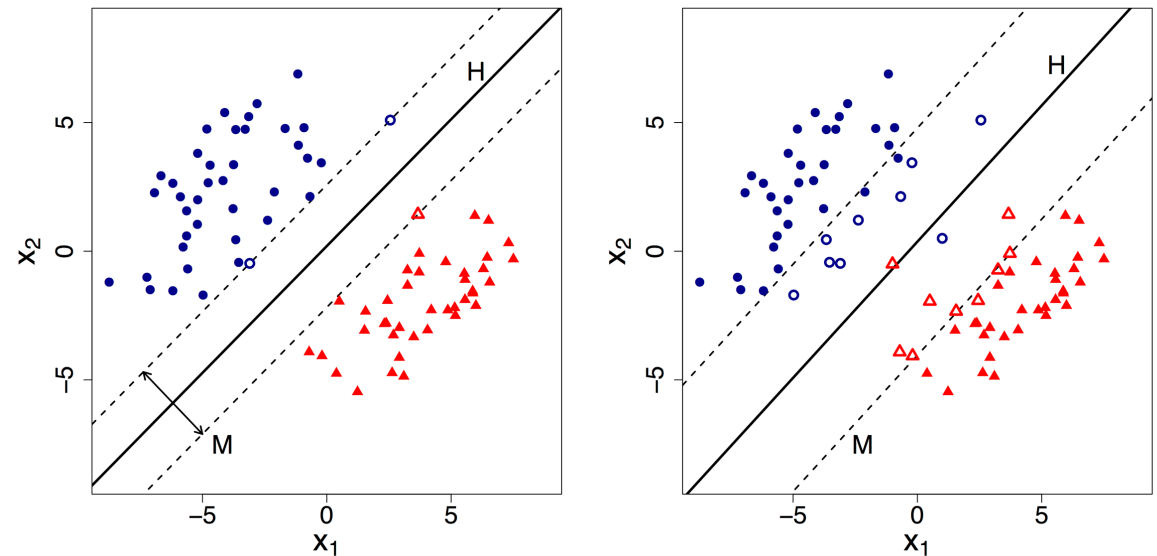
Intuitive Interpretation

- Why are fat separators preferable (good generalization performance)?
 - As the separator has a wider margin, there is a higher chance to correctly classify the test data that is similar to but slightly different from the training data!
 - Imagine a super narrow margin as below...

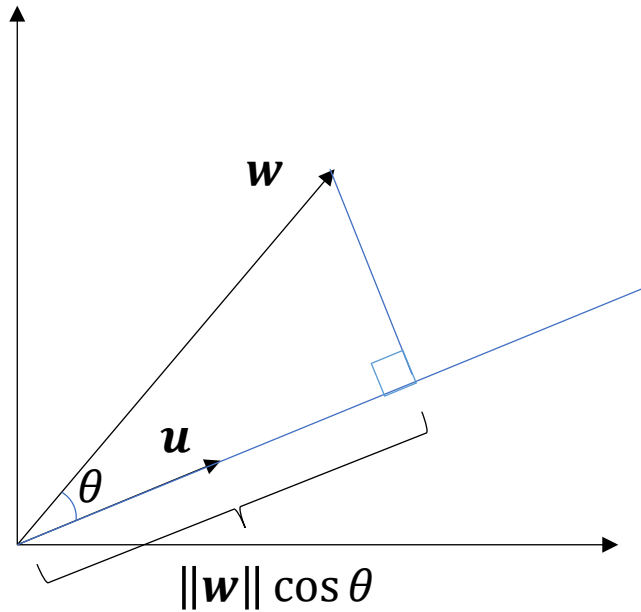


Even though this new sample is just next to the correctly classified training sample, it will be misclassified...

- Unfortunately, predictions can be wrong...
- **Support Vector Machine (SVM)**
 - What is it?
 - **Algorithm development**
- Kernel Method



Vector Inner Product



Why is this length $\|\mathbf{w}\| \cos \theta$?

$$\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} \quad \mathbf{u} = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

$$\|\mathbf{w}\| = \text{length}(\mathbf{w}) = \sqrt{w_1^2 + w_2^2}$$

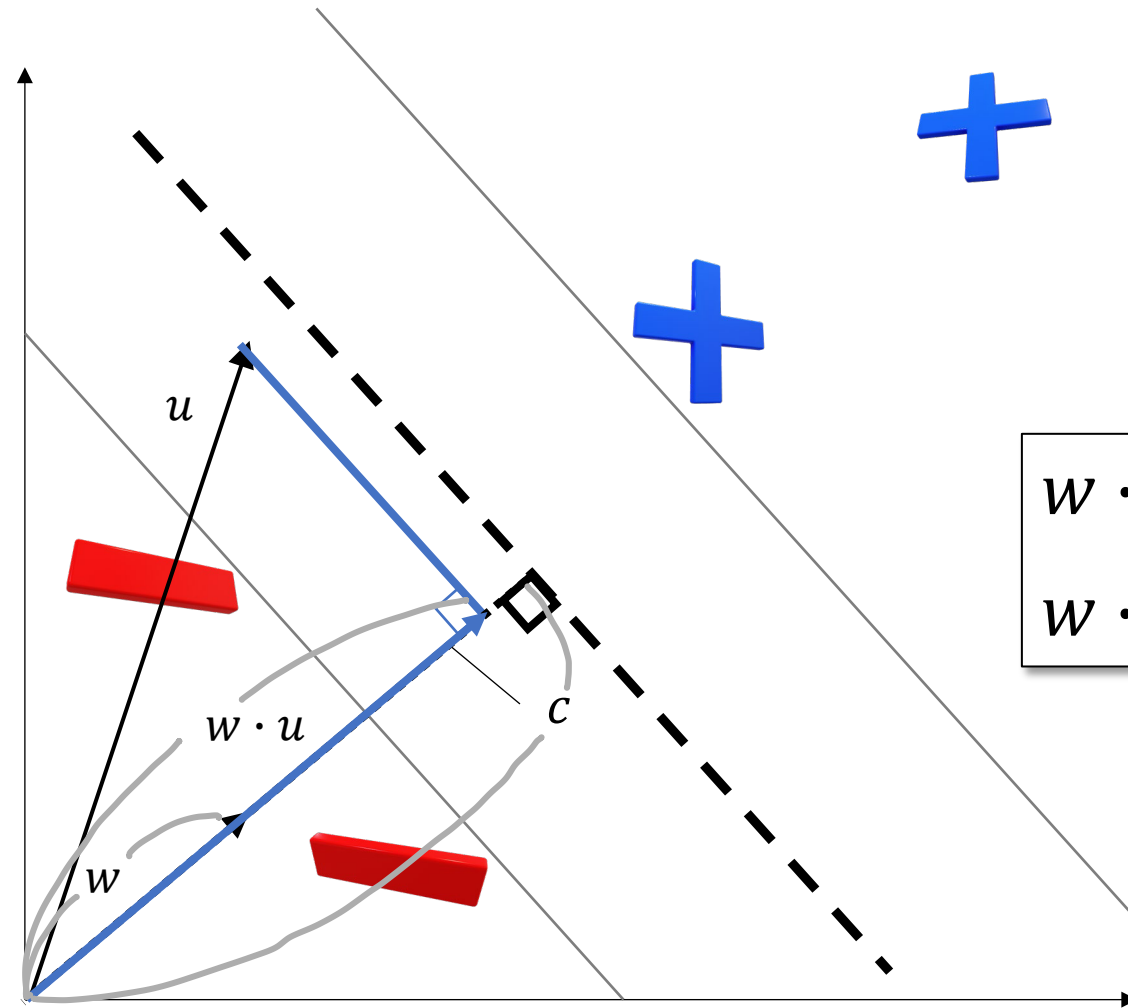
$$\mathbf{w}^T \mathbf{u} = \mathbf{u}^T \mathbf{w} = \mathbf{w} \cdot \mathbf{u}$$

$$= w_1 u_1 + w_2 u_2$$

$$= \|\mathbf{w}\| \|\mathbf{u}\| \cos \theta$$

Decision Rules (1/4)

1. Let us assume the middle line is drawn in a way that maximizes the street width.
2. Consider a unit vector w that is perpendicular to the middle line.
3. Say we have an unknown vector u .
4. By calculating the inner product of w and u , we can project u on w , which shows the distance from the origin to the median line.



Decision Rule

$$w \cdot u \geq c \quad c = -b$$

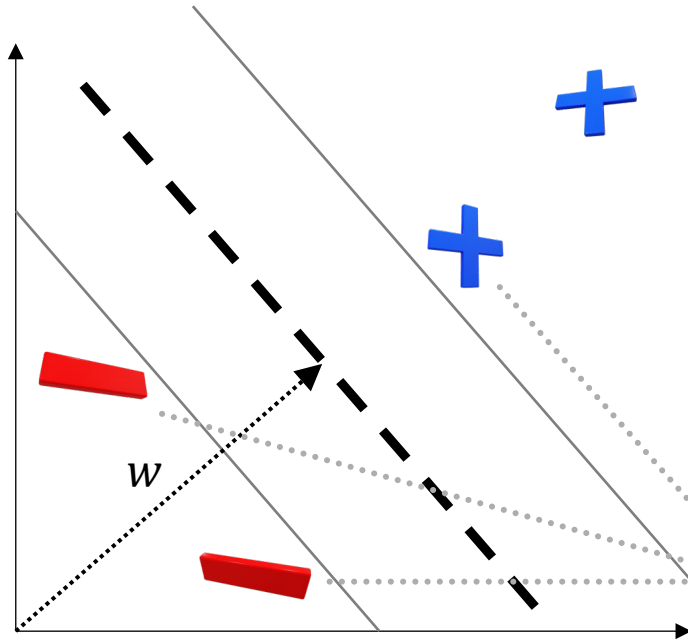
$$w \cdot u + b \geq 0 \text{ then, } +$$

Decision Rules (2/4)

Decision Rule

$$w \cdot u \geq c \quad c = -b$$

$w \cdot u + b \geq 0$ then, +



We want that $w \cdot x_+ + b \geq 1$ and

$$w \cdot x_- + b \leq -1$$

Let us introduce another variable y such that

label $\left(\begin{array}{l} y_i = 1 \text{ for } x_+ \text{ samples and} \\ y_i = -1 \text{ for } x_- \text{ samples.} \end{array} \right.$

By multiplying y_i to both sides, we get

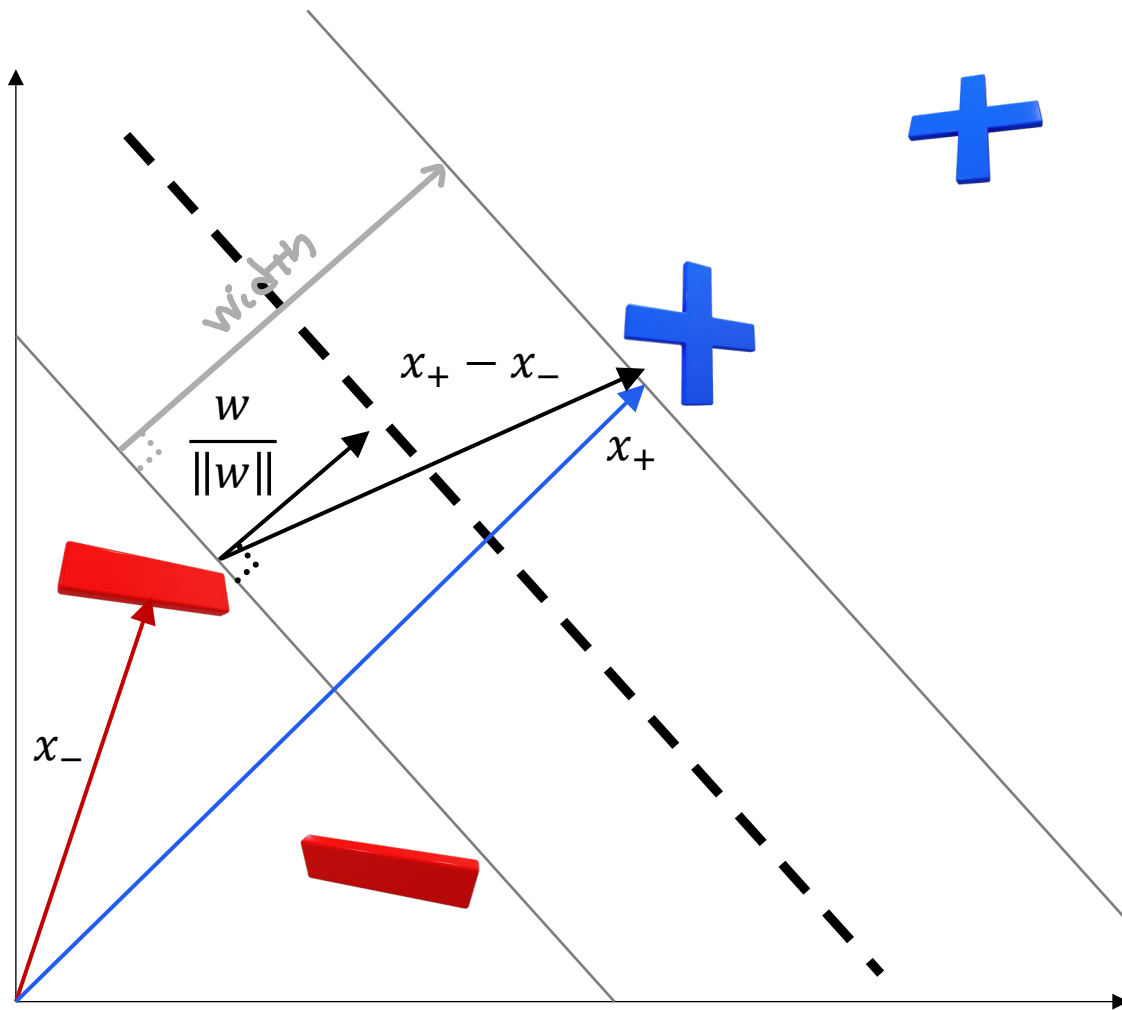
$$y_i(w \cdot x_i + b) \geq 1$$

$$y_i(w \cdot x_i + b) - 1 \geq 0$$

For the samples exactly on the boundary,

$$y_i(w \cdot x_i + b) - 1 = 0$$

Decision Rules (3/4)



Unit
perpendicular
vector

Width of the road is:

$$width = (x_+ - x_-) \cdot \frac{w}{\|w\|}$$

Remember what we derived:

$$y_i(w \cdot x_i + b) - 1 = 0$$

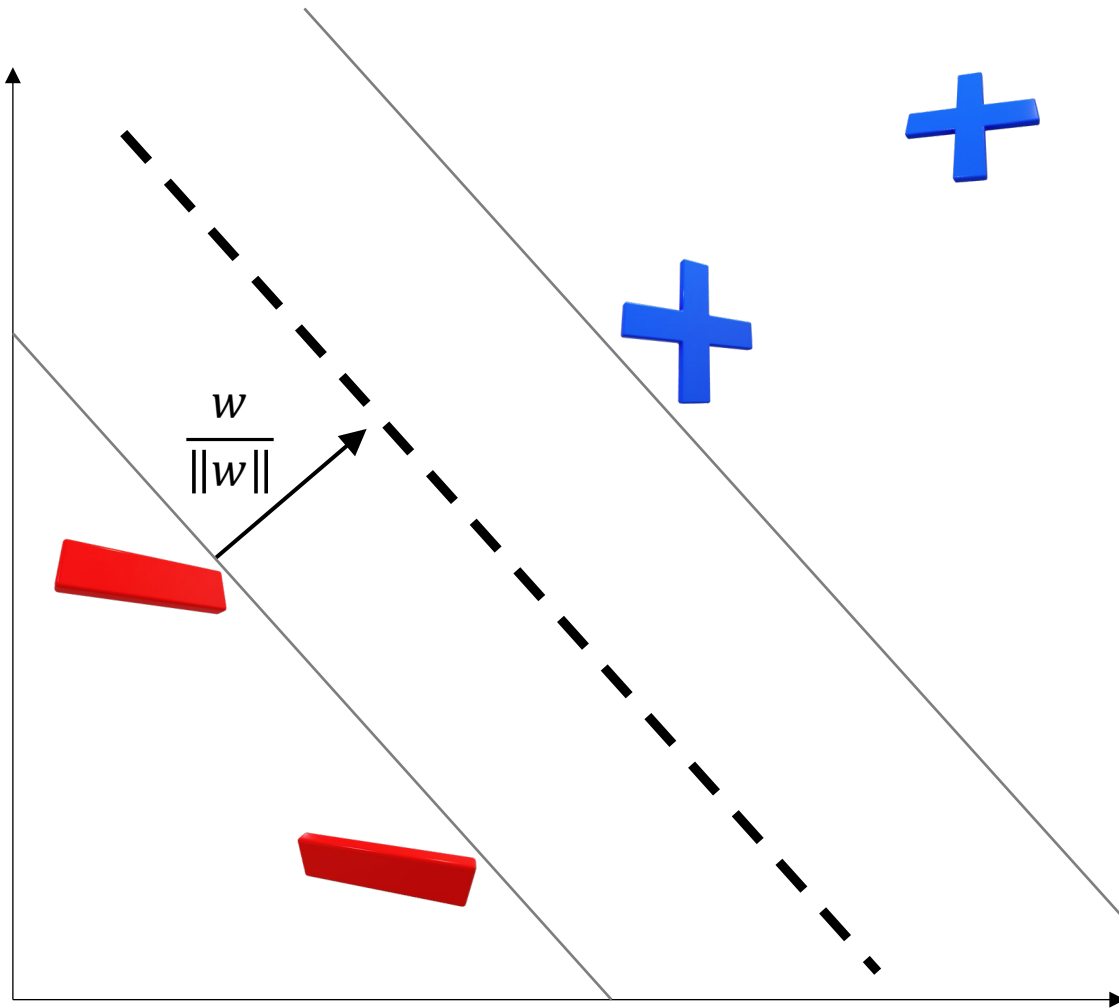
$$\text{For } x_+, (w \cdot x_i + b) - 1 = 0$$

$$\text{For } x_-, -(w \cdot x_i + b) - 1 = 0$$

Putting all together, we get:

$$\begin{aligned} width &= (1 - b + b + 1) \cdot \frac{1}{\|w\|} \\ &= \frac{2}{\|w\|} \leftarrow \text{원래 데이터에 영향 X} \end{aligned}$$

Decision Rules (4/4)



- Now we know the width of the road is $\frac{2}{\|w\|}$.
- So, our goal is:

$$\max \frac{2}{\|w\|} = \max \frac{1}{\|w\|} = \min \|w\|$$

$$\min \|w\| = \min \frac{1}{2} \|w\|^2$$

Why? Because it is mathematically convenient!

Lagrangian Primal-Dual (1/2)

Original Problem

$$\begin{aligned} &\text{Minimize } \frac{\|w\|^2}{2} \quad \text{목적} \\ &\text{Subject to } y_i(w^T x_i + b) - 1 \geq 0, \quad i = 1, 2, \dots, n \quad \text{제약조건} \end{aligned}$$

closed form solution X
GD 사용 X

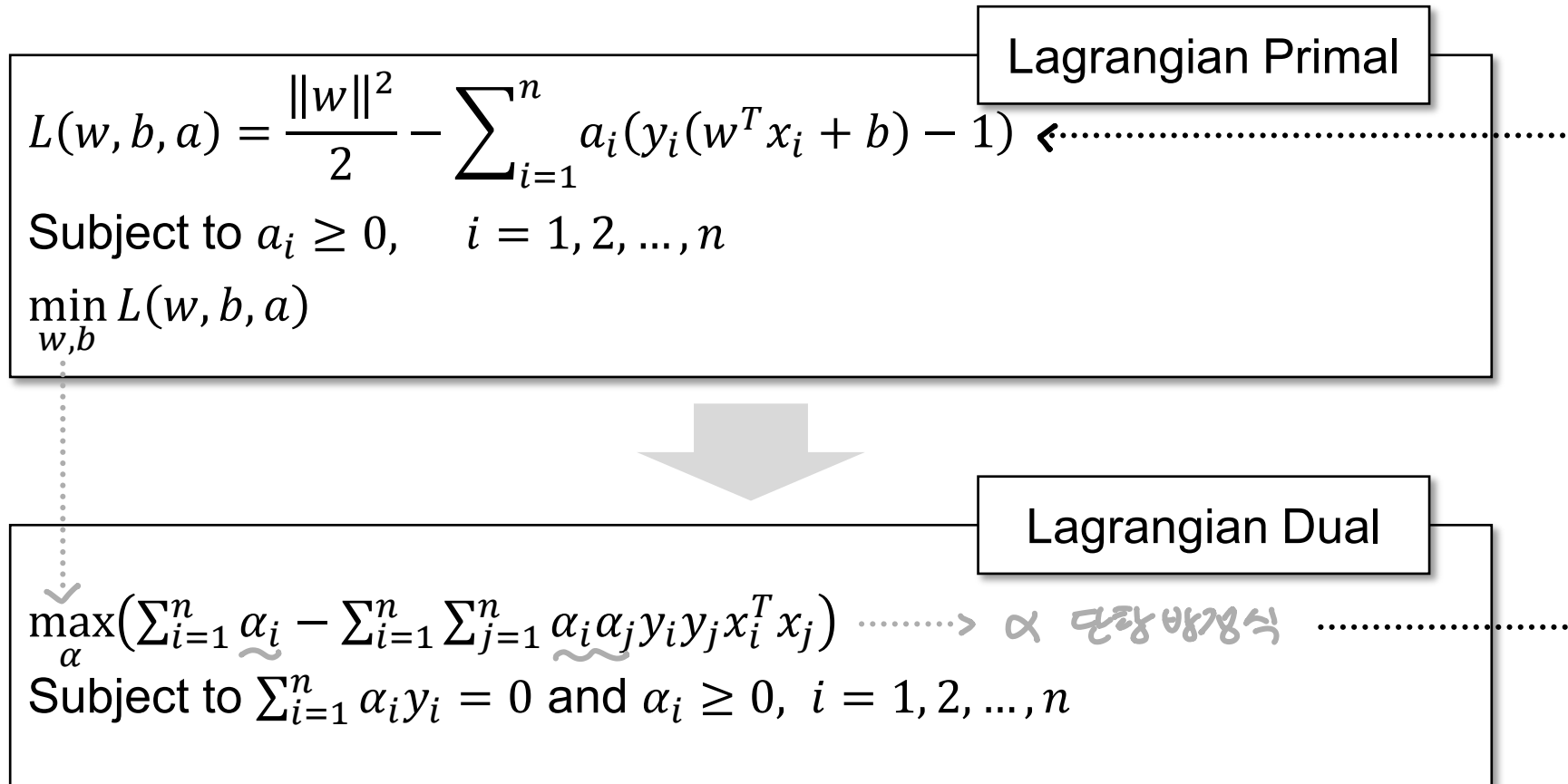
Lagrange multipliers

Lagrangian Primal

$$\begin{aligned} L(w, b, a) &= \frac{\|w\|^2}{2} - \sum_{i=1}^n a_i (y_i(w^T x_i + b) - 1) \\ &\text{Subject to } a_i \geq 0, \quad i = 1, 2, \dots, n \\ &\min_{w, b} L(w, b, a) \end{aligned}$$

제약조건을 Cost Function에 통합
: 제약조건에 i 별로 임의의 양수 변수를 곱한 식을 합한 것을 원래 Function에서 뺌

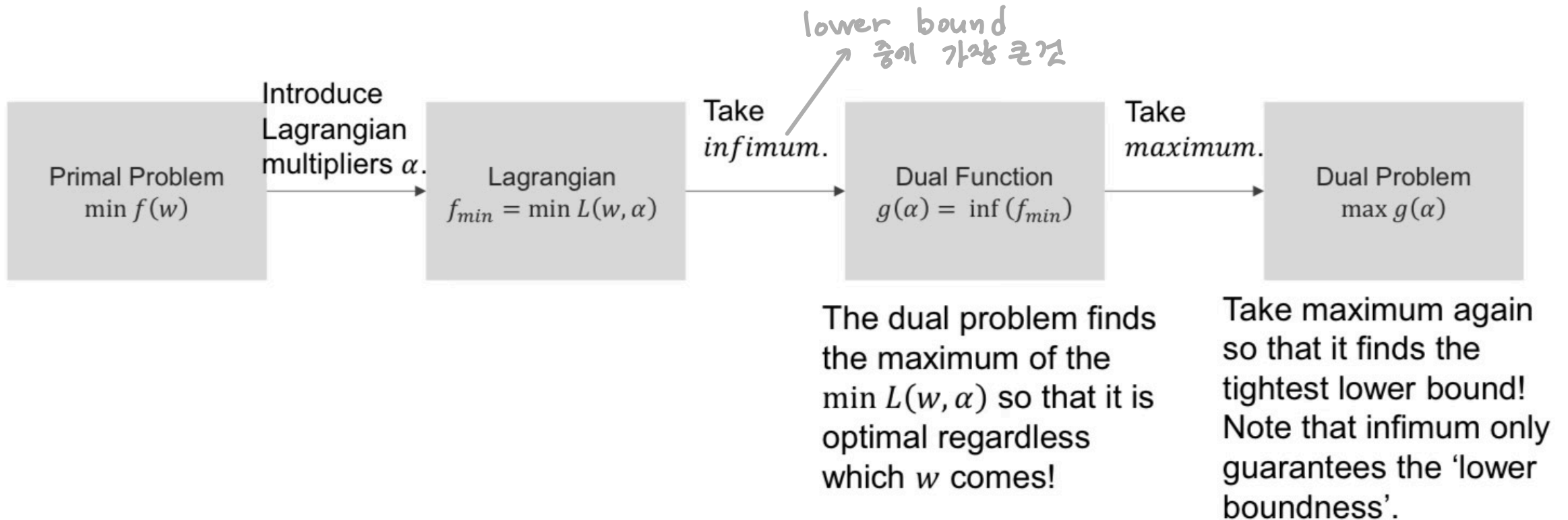
Lagrangian Primal-Dual (2/2)



α 의 해 대입

α 대입방정식

From Primal to Dual



How to Find Dual (1/3)

Lagrangian Primal

$$L(w, b, a) = \frac{\|w\|^2}{2} - \sum_{i=1}^n a_i (y_i (w^T x_i + b) - 1)$$

Subject to $a_i \geq 0, \quad i = 1, 2, \dots, n$

$$\min_{w, b} L(w, b, a)$$

L is a differentiable continuous function. So, the minimum is supposed to have a gradient of 0.

$$L = \frac{\|w\|^2}{2} - \sum_{i=1}^n a_i (y_i (w^T x_i + b) - 1)$$

$$\frac{\partial L}{\partial w} = w - \sum \alpha_i y_i x_i = 0 \quad \Rightarrow \quad w = \sum \alpha_i y_i x_i$$

$$\frac{\partial L}{\partial b} = - \sum \alpha_i y_i = 0 \quad \Rightarrow \quad \sum \alpha_i y_i = 0$$

How to Find Dual (2/3)

- Plug in the $w = \sum \alpha_i y_i x_i$ into our Lagrange equation.

$$\begin{aligned} L &= \frac{1}{2} \|w\|^2 - \sum \alpha_i [y_i (w \cdot x_i + b) - 1] \\ &= \frac{1}{2} \left(\sum \alpha_i y_i x_i \right) \cdot \left(\sum \alpha_j y_j x_j \right) - \sum \alpha_i \left[y_i \left(\left(\sum \alpha_j y_j x_j \right) \cdot x_i + b \right) - 1 \right] \\ &= \frac{1}{2} \left(\sum \alpha_i y_i x_i \right) \left(\sum \alpha_j y_j x_j \right) - \left(\sum \alpha_i y_i x_i \right) \left(\sum \alpha_j y_j x_j \right) - \sum \alpha_i y_i b + \sum \alpha_i \\ &= \sum \alpha_i - \frac{1}{2} \left(\sum \sum \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle \right) \quad \text{s.t.} \quad \sum \alpha_i y_i = 0 \end{aligned}$$

Now the loss consists only of α ! (No w and no b)

How to Find Dual (3/3)

$$\max_{\alpha} \left(L = \sum \alpha_i - \frac{1}{2} \left(\sum \sum \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle \right) \right)$$

Because the term with the highest order is negative, our optimization problem becomes maximization instead of minimization!

$$= \max_{\alpha} \left(\mathbf{1}^T \alpha - \frac{1}{2} \alpha^T \underbrace{\begin{bmatrix} y_1 y_1 \mathbf{x}_1^T \mathbf{x}_1 & y_1 y_2 \mathbf{x}_1^T \mathbf{x}_2 & \dots & y_1 y_m \mathbf{x}_1^T \mathbf{x}_m \\ y_2 y_1 \mathbf{x}_2^T \mathbf{x}_1 & y_2 y_2 \mathbf{x}_2^T \mathbf{x}_2 & \dots & y_2 y_m \mathbf{x}_2^T \mathbf{x}_m \\ \dots & \dots & \dots & \dots \\ y_n y_1 \mathbf{x}_n^T \mathbf{x}_1 & y_n y_2 \mathbf{x}_n^T \mathbf{x}_2 & \dots & y_n y_m \mathbf{x}_n^T \mathbf{x}_m \end{bmatrix}}_{\text{matrix}} \alpha \right) \quad \text{s.t.} \quad \sum \alpha_i y_i = 0$$

See that this matrix is already given (data and labels!).

자코비안 행렬

$$= \max_{\alpha} \left(\mathbf{1}^T \alpha - \frac{1}{2} \alpha^T \underset{\substack{\downarrow \\ \text{상수}}}{Q}} \alpha \right) \quad \text{s.t.} \quad \sum \alpha_i y_i = 0$$

Finally, we have Lagrangian Dual!

- It only consists of Lagrangian multipliers, α_i .
- It has become a maximization problem.
- The constraints are also much simpler than the primal.

Lagrangian Dual

$$\begin{aligned} & \max_{\alpha} \left(\sum_{i=1}^n \alpha_i - \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j \right) \\ & \text{Subject to } \sum_{i=1}^n \alpha_i y_i = 0 \text{ and } \alpha_i \geq 0, \quad i = 1, 2, \dots, n \end{aligned}$$

We Can Now Solve It Easily!

- Our Lagrangian dual, $L = \sum \alpha_i - \frac{1}{2} \left(\sum \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle \right)$, is a multi-variate quadratic equation.
 - **Quadratic Programming (QP)**: the process of solving certain mathematical optimization problems involving quadratic equations.
- There are many well-defined algorithms, such as Sequential Minimal Optimization (SMO), to solve quadratic programming problems.

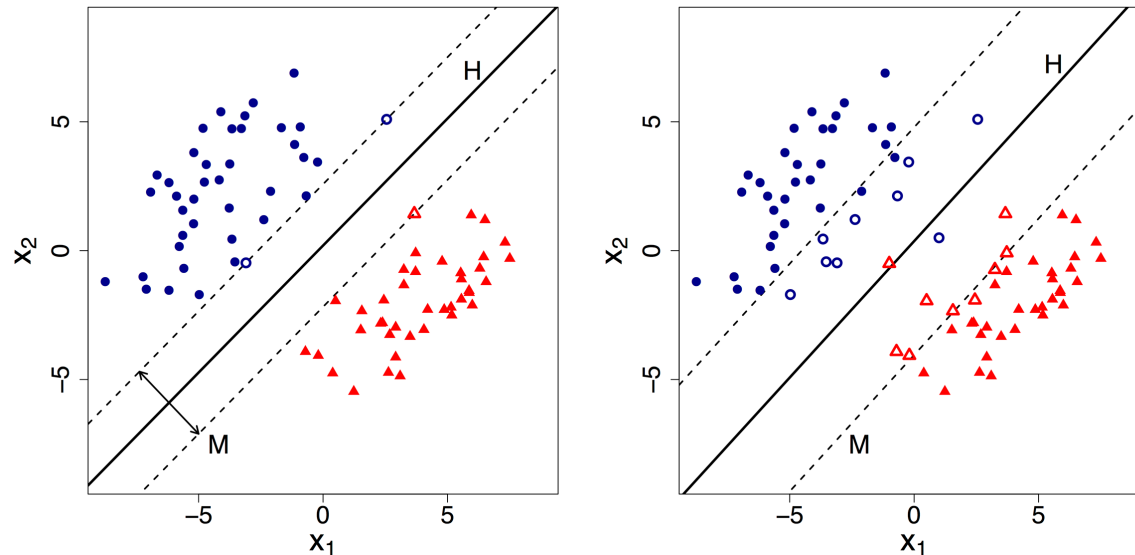
Margin 최대화 $\longrightarrow$ $\|w\|$ 최소화 $\longrightarrow$ 불완전 제약조건 $\longrightarrow$ Dual 문제로
변환 $\longrightarrow$ α 단방향 풀기 $\longrightarrow$ α 의 해 Primal 대입 $\longrightarrow$ 최적 $\|w\|$ 찾음

QP Gives Us α_i , Then...

- Once we find the optimal α_i , we can restore w using the below decision rule.
 - $w = \sum \alpha_i y_i x_i$
- Then, we can also find b by inserting w to the below decision rule.
 - $y_i(w \cdot x_i + b) - 1 = 0$
- Finally, we have found the optimal model parameters that provide the maximal margin (decision boundary).

SVM Model We Found

- Once we find w and b in the 2-dimensional problem space, they can be considered as slope and y-intercept, respectively.



Karush-Kuhn Tucker (KKT) Conditions

- Conditions for w, b, α are the optimal solution for Lagrangian Dual.

- Stationary: $\frac{\partial L}{\partial w} = w - \sum \alpha_i y_i x_i = 0$ and $\frac{\partial L}{\partial b} = -\sum \alpha_i y_i = 0$ 극점 이면서

- Primal feasibility: $y_i(w \cdot x_i + b) - 1 \geq 0$

- Dual feasibility: $\alpha_i \geq 0, i = 1, 2, \dots, n$

두 공간기서 모두 제약 조건 만족하면서

slack

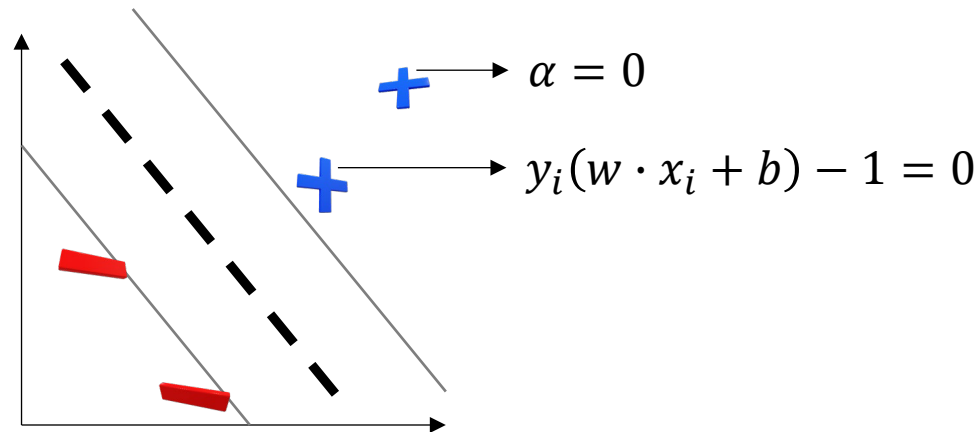
- * Complementary slackness: $\alpha_i(y_i(w \cdot x_i + b) - 1) = 0, i = 1, 2, \dots, n$

두 제약 조건 조건의 곱 = 0

Complementary Slackness Condition

- $\alpha_i(y_i(w \cdot x_i + b) - 1) = 0, \quad i = 1, 2, \dots, n$
 - For all the boundary points, the Lagrange multiplier α_i will not be 0.
 - $y_i(w \cdot x_i + b) - 1 = 0$, and regardless of α_i , the above condition is true for support vectors.
 - For all the internal points, the Lagrange multiplier α_i will be 0.
 - If $y_i(w \cdot x_i + b) - 1 \neq 0$, α_i should be 0 to satisfy the above condition.

두 제약 조건의 곱 = 0 형태를
통해 한 제약 조건만 만족해도
다른 쪽 조건을 만족하게 됨



Insight

- If we plug in what we found into the decision rule, we have

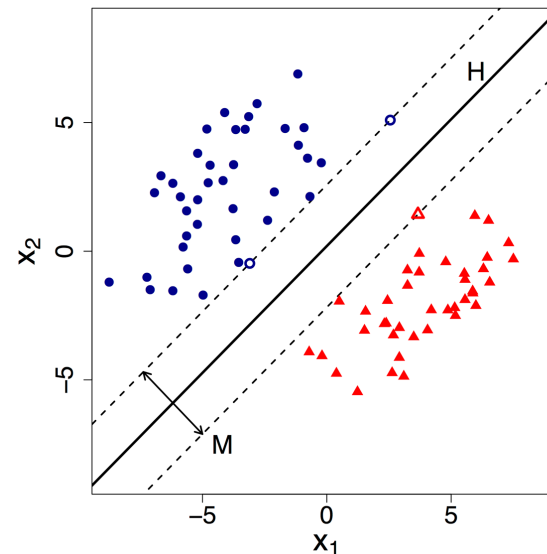
$$w = \sum \alpha_i y_i x_i \quad \rightarrow \quad \begin{array}{l} \text{Decision Rule} \\ w \cdot u \geq c \quad c = -b \\ w \cdot u + b \geq 0 \text{ then, } + \end{array}$$

$$\sum \alpha_i y_i \boxed{x_i \cdot u} + b \geq 0 \quad \text{then } +, \text{ for any } u$$

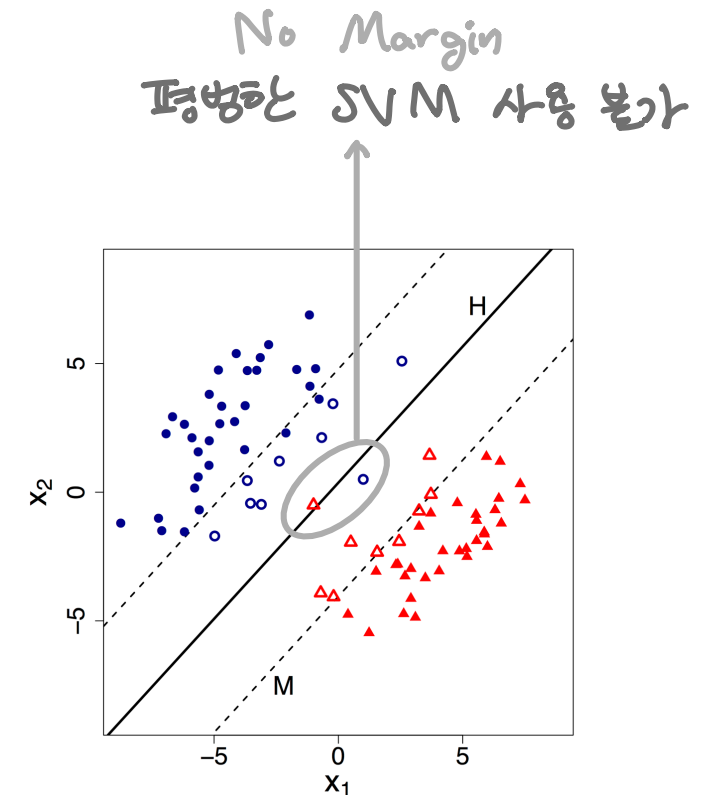
The whole system consists only of the inner-products of the input data vectors. This is a critical characteristic when the data is not linearly separable! We will talk about this later.

X는 항상 단독으로 쓰이지 않고 내적의 형태로만 쓰임

- Unfortunately, predictions can be wrong...
- Support Vector Machine (SVM)
 - What is it?
 - Algorithm development
- **Kernel Method**

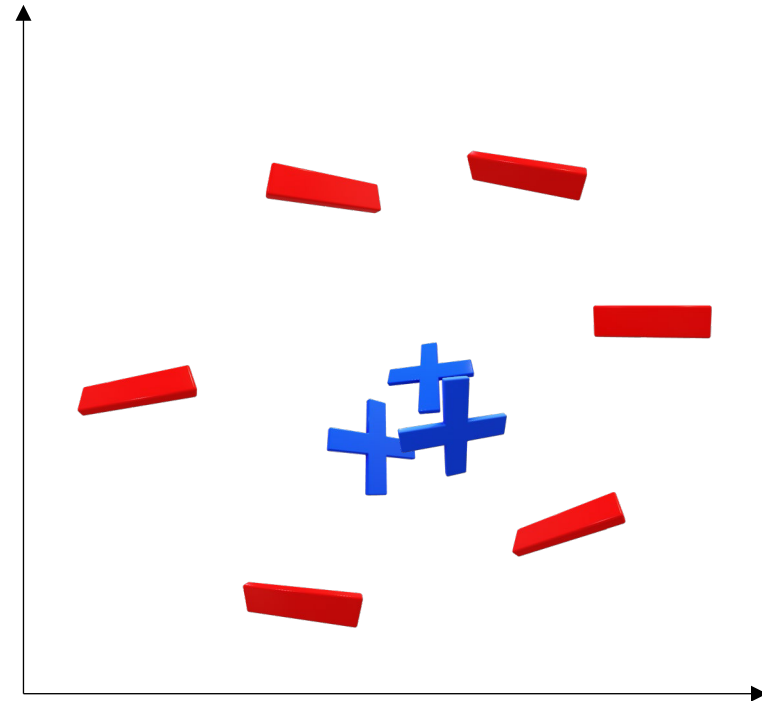
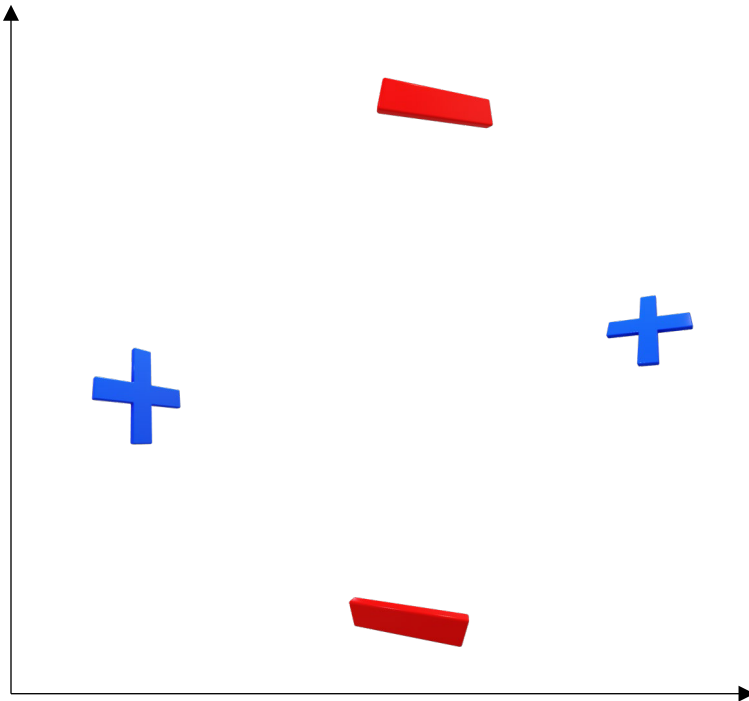


Linearly Separable

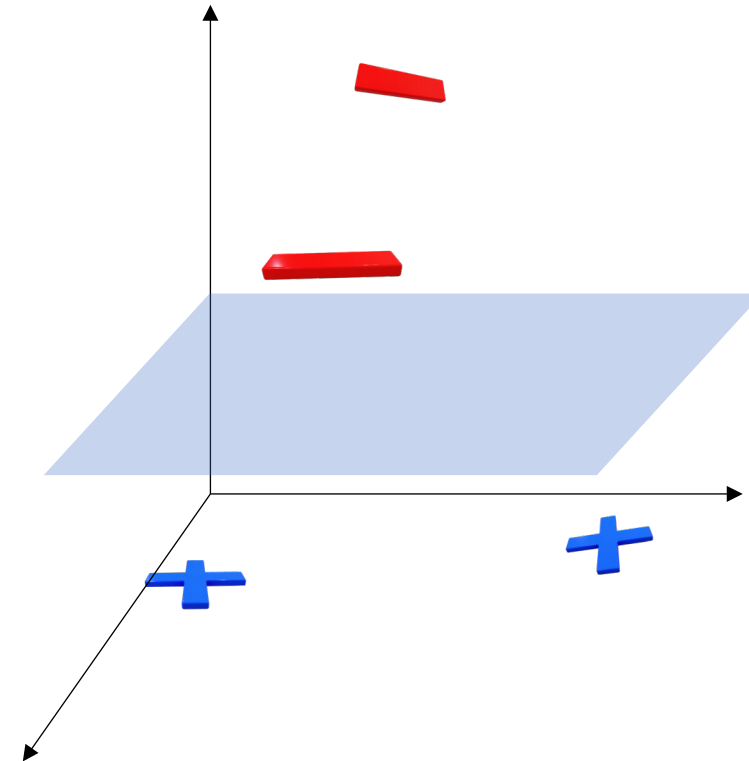
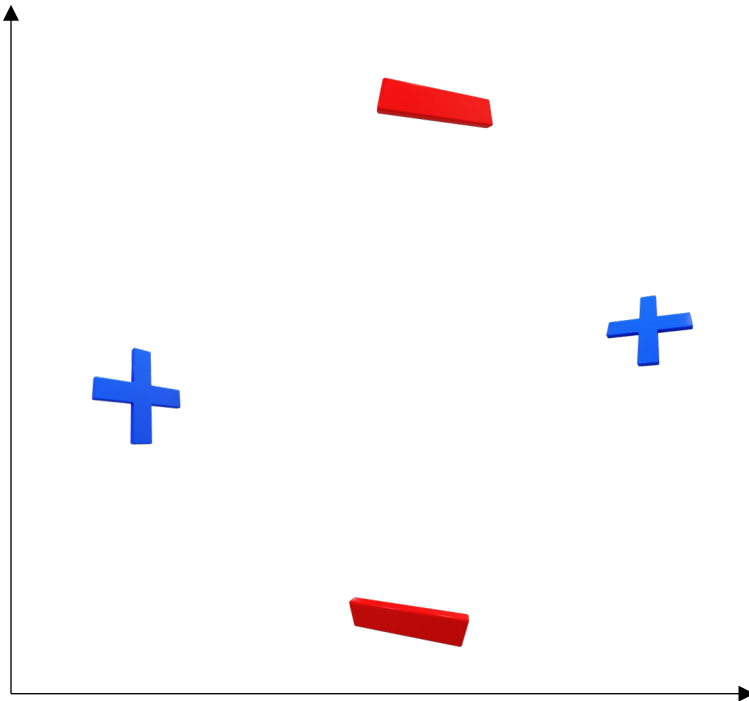


Not Linearly Separable

What If Data Are Not Linearly Separable?



Maybe, There would be More Information!



Transforming Data

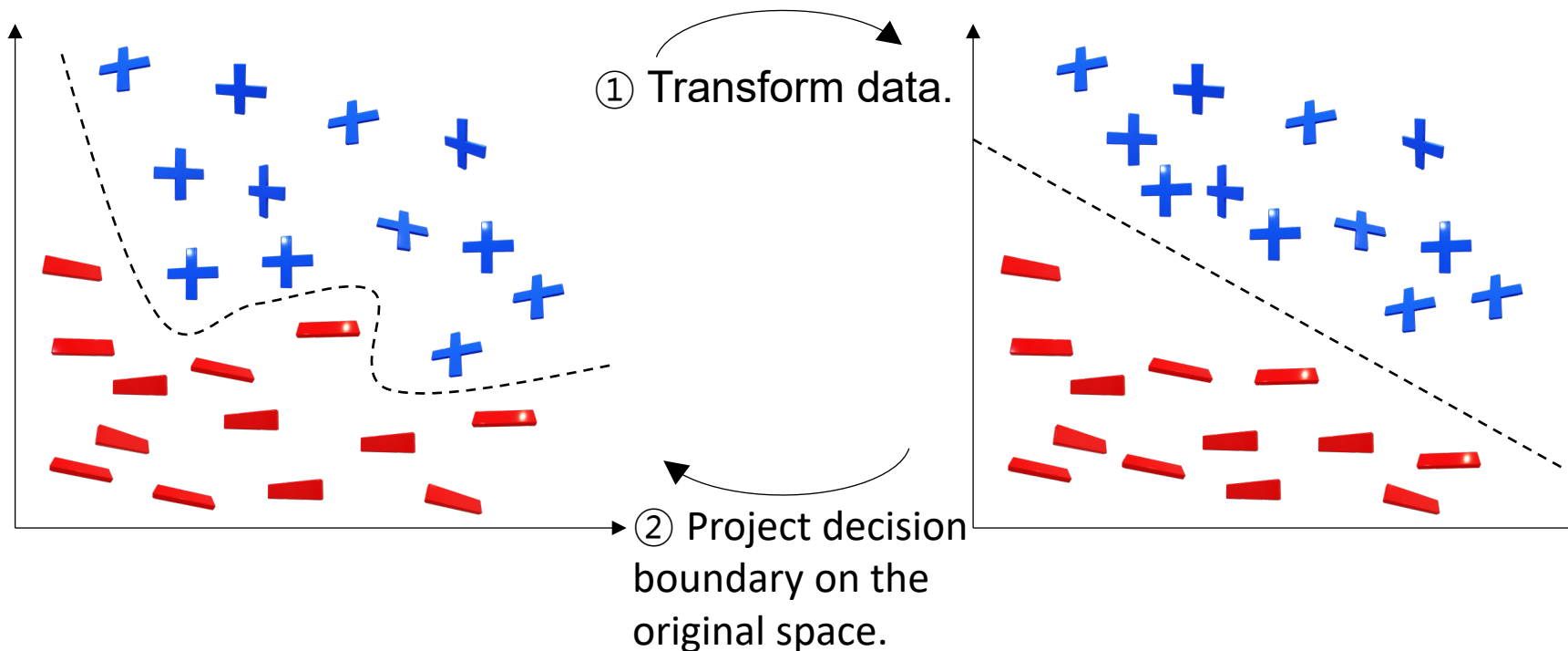
$$x = (x_1, x_2, \dots, x_n) \rightarrow \underbrace{\Phi(x)}_{p < q} \rightarrow z = (z_1, z_2, \dots, z_n)$$



Input space $x \in \mathbb{R}^p$



Feature space $z \in \mathbb{R}^q$



Data Mapping in SVM Lagrangian Dual

$$L = \sum \alpha_i - \frac{1}{2} \left(\sum \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle \right)$$

Take advantage of the fact that the dual consists only of the input data vectors!

$$L = \sum \alpha_i - \frac{1}{2} \left(\sum \alpha_i \alpha_j y_i y_j \langle \Phi(\mathbf{x}_i), \Phi(\mathbf{x}_j) \rangle \right)$$

X는 항상 내적의 형태로만 쓰이는 성질 이용

$$s.t. \quad \alpha_i \geq 0 \quad \forall i \quad \text{and} \quad \sum \alpha_i y_i = 0$$

Kernel: Efficient Mapping

$$L = \sum \alpha_i - \frac{1}{2} \left(\sum \alpha_i \alpha_j y_i y_j \langle \Phi(x_i), \Phi(x_j) \rangle \right)$$

전체 데이터의 차원을 개별로 올리는 작업

은 연산비용 매우 큼, 그러나 내적된 결과의

차원을 올리는 작업은 연산비용 절약가능

$$s.t. \quad \alpha_i \geq 0 \quad \forall i \quad \text{and} \quad \sum \alpha_i y_i = 0$$

- Note that $\Phi(x)$ values appear only in the form of $\langle \Phi(x_i), \Phi(x_j) \rangle$, not individually.
- So, instead of computing $\Phi(x)$ values for all individual data, we can directly compute the inner-product.
- We call it 'kernel'.

내적된 결과의 차원을 올리는 임의의 함수

$$K(x_i, x_j) = \langle \Phi(x_i), \Phi(x_j) \rangle$$

Kernels

$$\max_{\alpha} L = \max_{\alpha} \left(\sum \alpha_i - \frac{1}{2} \left(\sum \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle \right) \right) \quad \text{Not Linearly Separable}$$

$$\max_{\alpha} \left(\sum \alpha_i - \frac{1}{2} \left(\sum \alpha_i \alpha_j y_i y_j \langle \Phi(x_i), \Phi(x_j) \rangle \right) \right) \quad \text{Linearly Separable}$$

but 계산 비용 ↑

$$\max_{\alpha} \left(\sum \alpha_i - \frac{1}{2} \left(\sum \alpha_i \alpha_j y_i y_j K(x_i, x_j) \right) \right) \quad \text{Linearly Separable}$$

계산 비용 ↓

s. t. $\alpha_i \geq 0 \forall i$ and $\sum \alpha_i y_i = 0$

Kernels look like an inner-product!

- A kernel can be considered as a function that calculates the inner-product in a higher dimensional space.
- Mercer's Theorem (reference [here](#))
 - If a function is symmetric and positive semi-definite, it is a valid kernel.
 - That is, $K(x_i, x_j) = \Phi(x_i)^T \Phi(x_j) = x_i^T A^T A x_j =$ a certain positive scalar or zero!

Kernel Trick

“Given an algorithm which is formulated in terms of a symmetric and positive semi-definite kernel K_1 , one can construct an alternative algorithm by replacing K_1 with another symmetric and positive semi-definite kernel K_2 .”

**You can find any positive definite kernels and
plug in them to SVM!**



Common Kernels

- Polynomials of degree exactly d : monomials with degree of d

$$K(u, v) = (u \cdot v)^d$$

- Polynomials of degree up to d : monomials with degree of up to d

$$K(u, v) = (u \cdot v + 1)^d$$

- Sigmoid kernel

$$K(u, v) = \tanh(u \cdot v + 1)^d$$

- Gaussian kernel

$$K(u, v) = \exp\left(-\frac{\|u - v\|^2}{2\sigma^2}\right)$$

- And many others (very active area of research!)

Example: Polynomial Kernel (1/4)

- Let $x_i \in \mathbb{R}^2$ such as $(x_{i,1}, x_{i2})$.
- Consider two data samples, $(x_{i,1}, x_{i2})$ and $(x_{j,1}, x_{j2})$.
- Then, a polynomial kernel can be defined as follows.

$$\begin{aligned} K(x_i, x_j) &= \langle x_i, x_j \rangle^2 \\ &= (x_{i1}x_{j1} + x_{i2}x_{j2})^2 \\ &= (x_{i1}^2x_{j1}^2 + x_{i2}^2x_{j2}^2 + 2x_{i1}x_{i2}x_{j1}x_{j2}) \\ &= \langle \Phi(x_i), \Phi(x_j) \rangle, \quad \text{where } \Phi(x_i) = (x_{i1}^2, x_{i2}^2, \sqrt{2}x_{i1}x_{i2}) \\ &\quad \Phi(x_j) = (x_{j1}^2, x_{j2}^2, \sqrt{2}x_{j1}x_{j2}) \end{aligned}$$

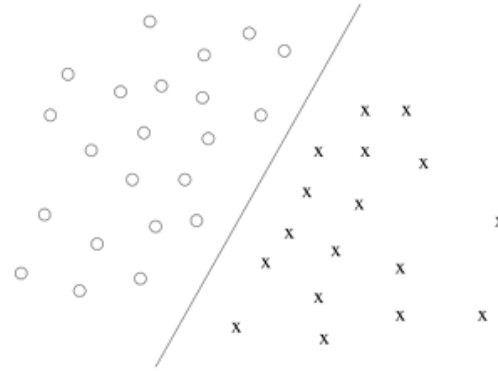
See that taking a dot product and exponentiating gives the same results as mapping into high dimensional space and then taking a dot product.

Note that $\Phi(x_i) \in \mathbb{R}^3$

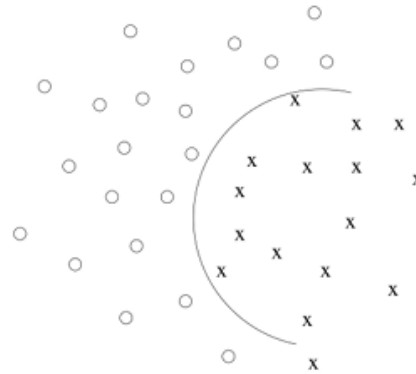
Example: Polynomial Kernel (2/4)

- We saw that, given by $K(x_i, x_j) = \langle x_i, x_j \rangle^d$, $\Phi(x)$ contains monomials of degree d .
- Real-world example of visual pattern recognition:
 - 16×16 pixel images
 - $\Phi(x)$ has 10^{10} monomials of degree 5.
 - However, the kernel trick allows us to ***avoid that expensive calculations!***
 - Instead of calculating all monomials, $x_{i1}^2, x_{i2}^2, \sqrt{2}x_{i1}x_{i2}$, separately, we could directly calculate the kernel $(x_{i1}x_{j1} + x_{i2}x_{j2})^2$.

Example: Polynomial Kernel (3/4)



Linear separator in the feature ϕ -space



Non-linear separator in the original x -space

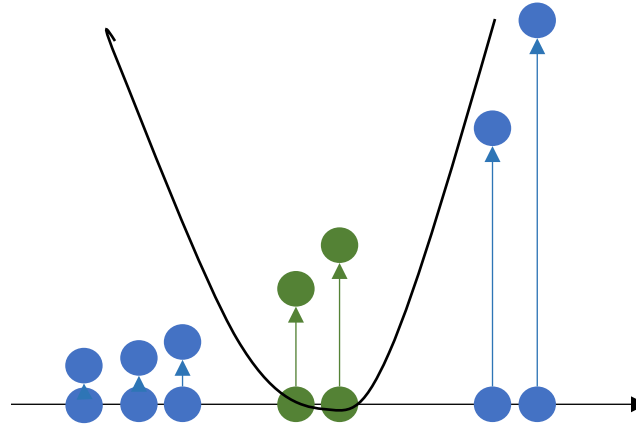
[Tommi Jaakkola]

Example: Polynomial Kernel (4/4)

Given the input data $x \in \mathbb{R}$, let us consider the polynomial kernel with $d = 2$.

$$\begin{aligned} K(x_i, x_j) &= (\langle x_i, x_j \rangle + 1)^2 = (x_i x_j + 1)^2 \\ &= x_i^2 x_j^2 + 2x_i x_j + 1 \\ &= (\sqrt{2}x_i, x_i^2, 1) \cdot (\sqrt{2}x_j, x_j^2, 1) \end{aligned}$$

x_i has been moved to higher dimensional space, $(\sqrt{2}x_i, x_i^2, 1)$!



Example: Gaussian Kernel (1/3)

- Also called *Radial Basis Function* (RBF) kernel.

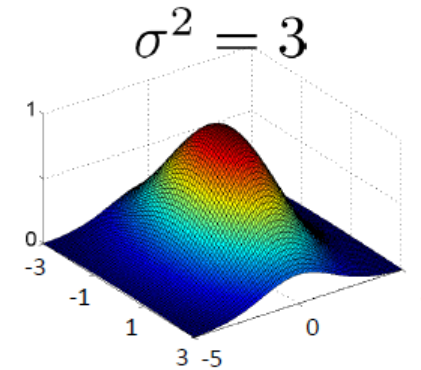
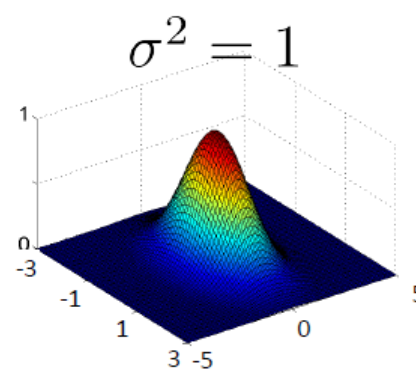
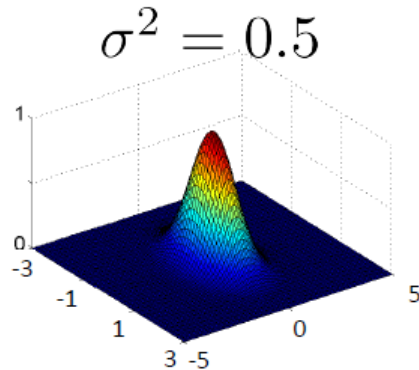
$$K(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right) \longrightarrow$$

x_i 와 x_j 의 거
리가 멀수록, 거
리가 가까우면 0
값 ↑ , 멀면 0 ↓

- Has value of 1 when $x_i = x_j$.
- The value falls off to 0 as the distance increases.
- The σ^2 is a tunable hyper-parameter.

↓

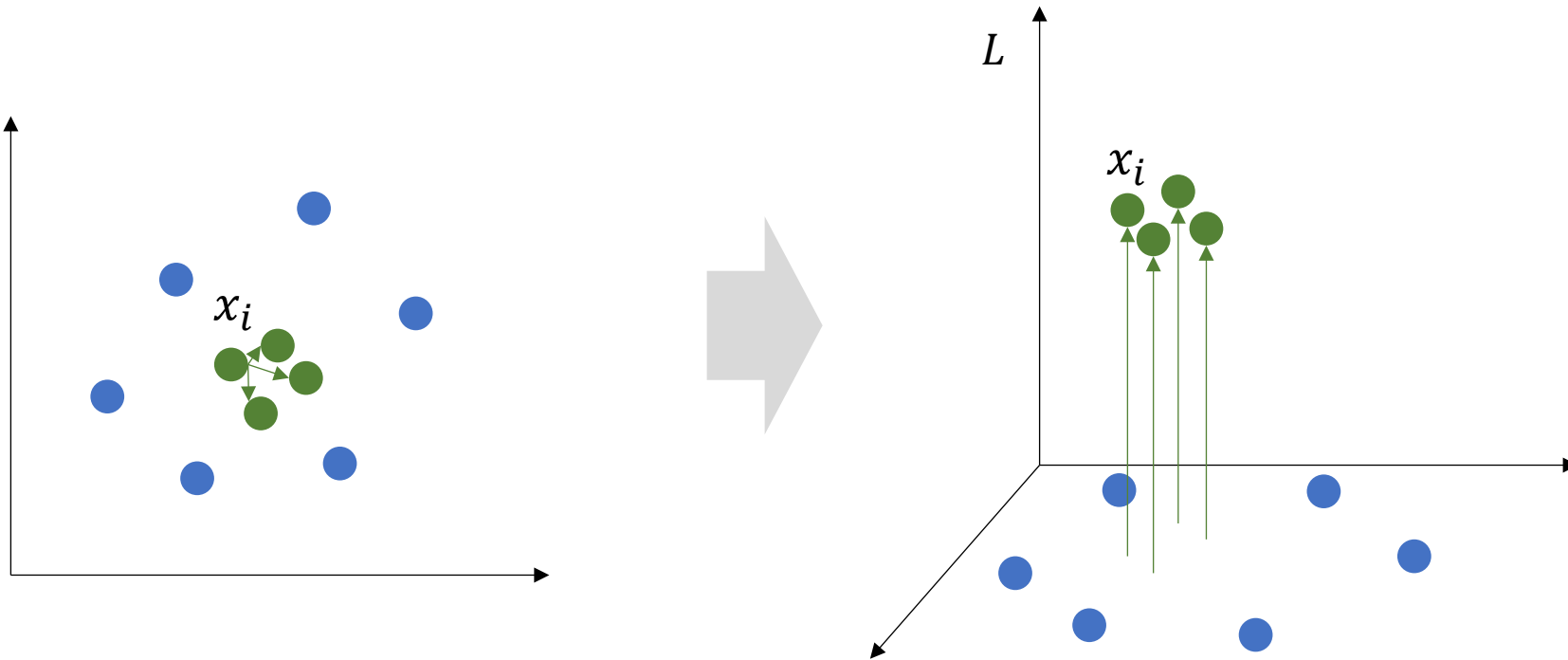
Linearly Separable



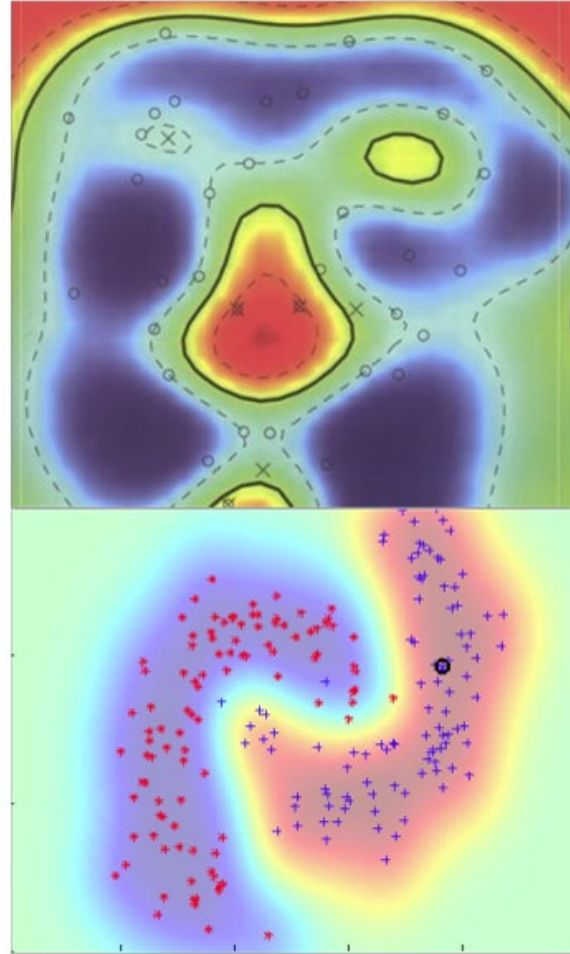
Example: Gaussian Kernel (2/3)

- If x_i is close to other points, the $K(x_i, x_j)$ value will be close to 1.
- If x_i is far from other points, the $K(x_i, x_j)$ value will go towards 0.

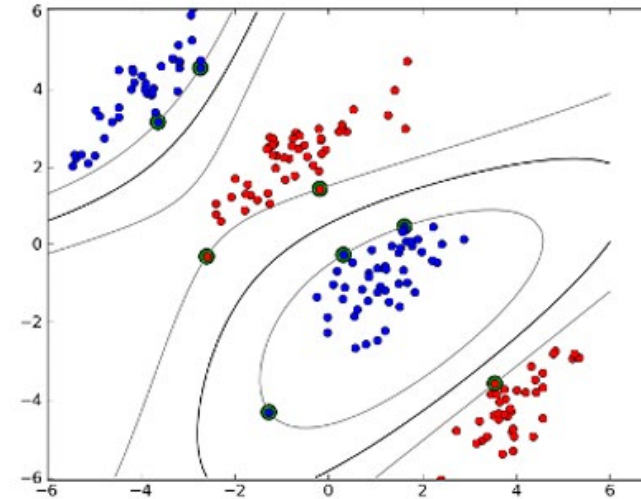
$$K(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$$



Example: Gaussian Kernel (3/4)

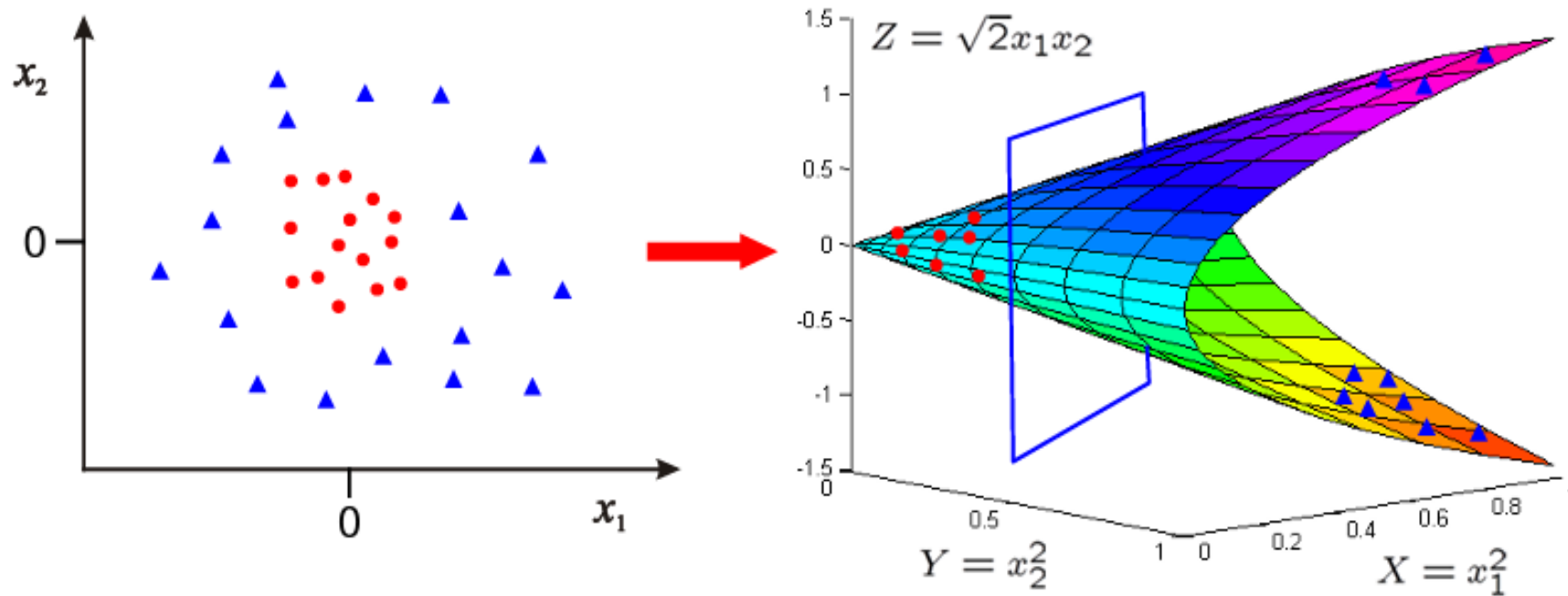


[Cynthia Rudin]



[mblondel.org]

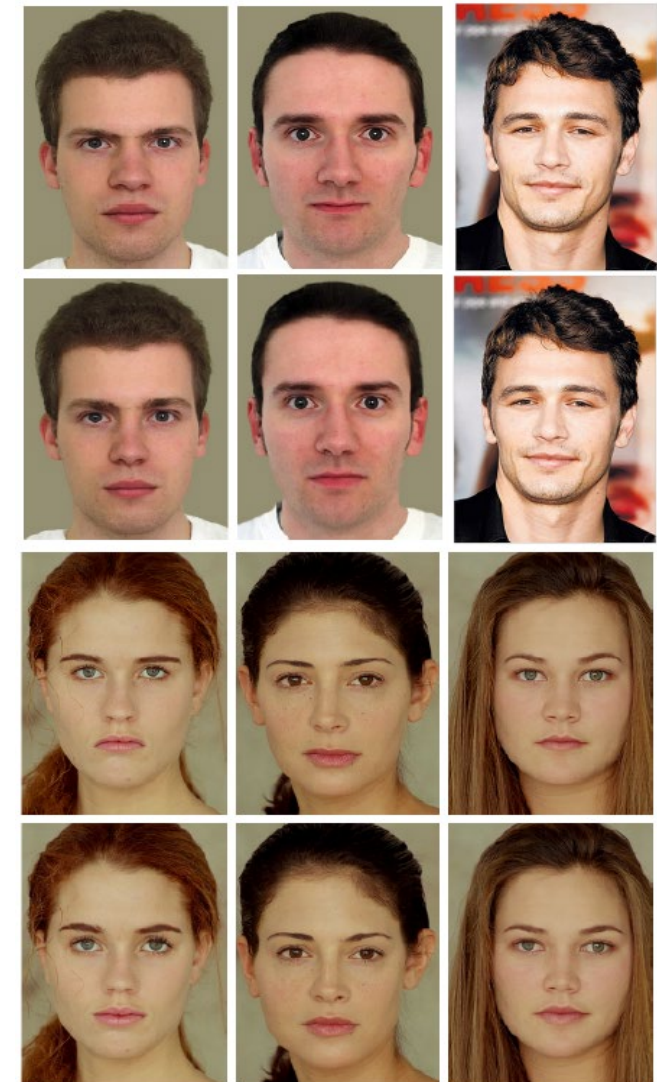
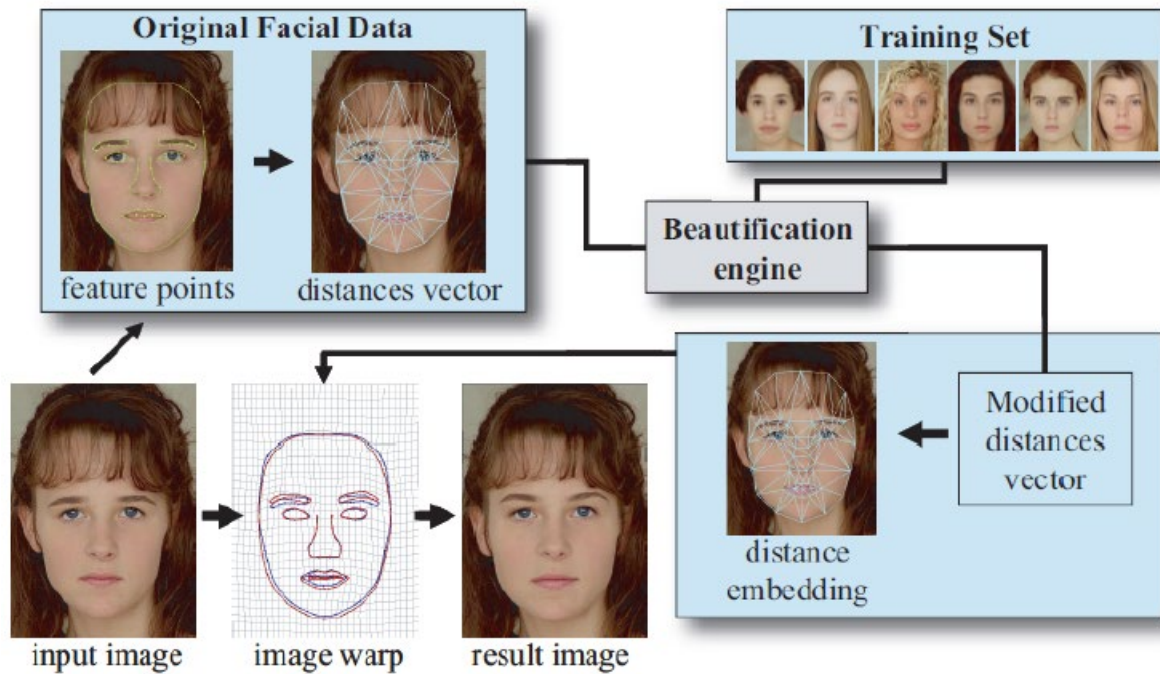
Example: Gaussian Kernel (4/4)



Choosing Kernel Functions

- SVM itself is a solid ML technique. The thing is how to choose a good kernel for your problem.
- In general, Gaussian kernel can be a good starting point.
- Remember, you should find the best kernel or even design a good one based on a deep understanding of your own data.

Applications: Automatic Photo Retouching



Notes

- We haven't deeply talked about the following things...
 - Primal-Dual Problem
 - Quadratic Programming
- If you really want to fully understand those techniques, study 'convex optimization'.
 - You can easily find many $\times 3$ articles online!

Any Questions?

SVM with kernel method is one of the most powerful ML methods!

